Python basic

Asia Quant Academy

Version 2026

Agenda

- 1 Introduction
- 2 Installing Python
- 3 Installing IDE: Visual Studio Code / PyCharm
- 4 Python Basic
- 5 List Comprehension
- 6 Native Datatypes
- 7 Object Oriented Programming (OOP):
- 8 Python PYPI

Magic Methods

In Python, special methods are also called magic methods, or dunder methods. This latter terminology, dunder, refers to a particular naming convention that Python uses to name its special methods and attributes.

Operator	Supporting Method
+	`.__add__(self, other)`
-	`.__sub__(self, other)`
*	`.__mul__(self, other)`
/	`.__truediv__(self, other)`
//	`.__floordiv__(self, other)`
%	`.__mod__(self, other)`
**	`.__pow__(self, other[, modulo])`

Operator	Supporting Method
<	`.__lt__(self, other)`
<=	`.__le__(self, other)`
==	`.__eq__(self, other)`
!=	`.__ne__(self, other)`
>=	`.__ge__(self, other)`
>	`.__gt__(self, other)`

Method	Description
`.__getitem__()`	Called when you access an item using indexing like in `sequence[index]`
`.__len__()`	Called when you invoke the built-in `len()` function to get the number of items in the underlying sequence
`.__contains__()`	Called when you use the sequence in a membership test with the `in` or `not in` operator
`.__reversed__()`	Called when you use the built-in `reversed()` function to get a reversed version of the underlying sequence

Context Managers

- Context managers allow you to allocate and release resources precisely when you want to. The most widely used example of context managers is the with statement. Suppose you have two related operations which you'd like to execute as a pair, with a block of code in between. Context managers allow you to do specifically that. For example:

```
with open('some_file', 'w') as opened_file:
opened_file.write('Hola!')
```

- The above code opens the file, writes some data to it and then closes it. If an error occurs while writing the data to the file, it tries to close it. The above code is equivalent to:

```
file = open('some_file', 'w')
try:file.write('Hola!')
finally:file.close()
```

- While comparing it to the first example we can see that a lot of boilerplate code is eliminated just by using with. The main advantage of using a with statement is that it makes sure our file is closed without paying attention to how the nested block exits.
- A common use case of context managers is locking and unlocking resources and closing opened files (as I have already shown you).
- Let's see how we can implement our own Context Manager. This should allow us to understand exactly what's going on behind the scenes.

Implementing a Context Manager as a Class

- At the very least a context manager has an __enter__ and __exit__ method defined. Let's make our own file-opening Context Manager and learn the basics.

```
class File(object):
def __init__(self, file_name, method):self.file_obj = open(file_name,
method)
def __enter__(self):return self.file_obj
def __exit__(self, type, value,     traceback):
      self.file_obj.close()
```

- Just by defining __enter__ and __exit__ methods we can use our new class in a with statement. Let's try:

```
with File('demo.txt', 'w') as opened_file:
      opened_file.write('Hola!')
```

Implementing a Context Manager as a Class

- Our __exit__ method accepts three arguments. They are required by every __exit__ method which is a part of a Context Manager class. Let’s talk about what happens under-the-hood.

1.The with statement stores the __exit__ method of the File class.

2.It calls the __enter__ method of the File class.

3.The __enter__ method opens the file and returns it.

4.The opened file handle is passed to opened_file.

5.We write to the file using .write().

6.The with statement calls the stored __exit__ method.

7.The __exit__ method closes the file.

Handling Exceptions

- We did not talk about the type, value and traceback arguments of the __exit__ method. Between the 4th and 6th step, if an exception occurs, Python passes the type, value and traceback of the exception to the __exit__ method. It allows the __exit__ method to decide how to close the file and if any further steps are required. In our case we are not paying any attention to them.

- What if our file object raises an exception? We might be trying to access a method on the file object which it does not supports. For instance:

```
with File('demo.txt', 'w') as
opened_file:opened_file.undefined_function('Hola!')
```

Let's list the steps which are taken by the with statement when an error is encountered:

1.It passes the type, value and traceback of the error to the __exit__ method.

2.It allows the __exit__ method to handle the exception.

3.If __exit__ returns True then the exception was gracefully handled.

4.If anything other than True is returned by the __exit__ method then the exception is raised by the with statement.

Handling Exceptions

In our case the __exit__ method returns None (when no return statement is encountered then the method returns None). Therefore, the with statement raises the exception:

```
Traceback (most recent call last):
File "<stdin>", line 2, in <module>
AttributeError: 'file' object has no attribute 'undefined_function'
```

Let’s try handling the exception in the __exit__ method:

```
class File(object):
def __init__(self, file_name, method):self.file_obj = open(file_name, method)
def __enter__(self):return self.file_obj
def __exit__(self, type, value, traceback):print("Exception has been
handled")self.file_obj.close()return True


with File('demo.txt', 'w') as opened_file:opened_file.undefined_function()
```

Handling Exceptions

Our __exit__ method returned True, therefore no exception was raised by the with statement.

This is not the only way to implement Context Managers. There is another way and we will be looking at it in the next section.

Implementing a Context Manager as a Generator

We can also implement Context Managers using decorators and generators. Python has a contextlib module for this very purpose.

Instead of a class, we can implement a Context Manager using a generator function. Let's see a basic, useless example:

```
from contextlib import contextmanager
@contextmanager
def open_file(name):
    f = open(name, 'w')
    try:
        yield f
    finally:
        f.close()
```

Okay! This way of implementing Context Managers appear to be more intuitive and easy. However, this method requires some knowledge about generators, yield and decorators. In this example we have not caught any exceptions which might occur. It works in mostly the same way as the previous method.

Implementing a Context Manager as a Generator

Let’s dissect this method a little.

1.Python encounters the yield keyword. Due to this it creates a generator instead of a normal function.

2.Due to the decoration, contextmanager is called with the function name (open_file) as its argument.

3.The contextmanager decorator returns the generator wrapped by the GeneratorContextManager object.

4.The GeneratorContextManager is assigned to the open_file function. Therefore, when we later call the open_file function, we are actually calling the GeneratorContextManager object.

So now that we know all this, we can use the newly generated Context Manager like this:

```
with open_file('some_file') as f:
    f.write('hola!')
```

Lambdas

Lambdas are one line functions. They are also known as anonymous functions in some other languages. You might want to use lambdas when you don’t want to use a function twice in a program. They are just like normal functions and even behave like them.

Blueprint

```
lambda argument: manipulate(argument)
```

Example

```
add = lambda x, y: x + y
print(add(3, 5))
# Output: 8
```

Lambdas

Here are a few useful use cases for lambdas and just a few ways in which they are used in the wild:

List sorting

```
a = [(1, 2), (4, 1), (9, 10), (13, -3)]
a.sort(key=lambda x: x[1])
print(a)
# Output: [(13, -3), (4, 1), (1, 2), (9, 10)]
```

Parallel sorting of lists

```
data = zip(list1, list2)
data = sorted(data)
list1, list2 = map(lambda t: list(t), zip(*data))
```

Decorators

Decorators are a significant part of Python. In simple words: they are functions which modify the functionality of other functions. They help to make our code shorter and more Pythonic. Most beginners do not know where to use them so I am going to share some areas where decorators can make your code more concise.

First, let’s discuss how to write your own decorator.

It is perhaps one of the most difficult concepts to grasp. We will take it one step at a time so that you can fully understand it.

Everything in Python is an object:

First of all let’s understand functions in Python:

```
def hi(name="yasoob"):
    return "hi " + name

print(hi())
# output: 'hi yasoob'

# We can even assign a function to a variable like
greet = hi
# We are not using parentheses here because we are not calling the function hi
# instead we are just putting it into the greet variable. Let's try to run this

print(greet())
# output: 'hi yasoob'

# Let's see what happens if we delete the old hi function!
del hi
print(hi())
#outputs: NameError

print(greet())
#outputs: 'hi yasoob'
```

Defining functions within functions:

So those are the basics when it comes to functions. Let's take your knowledge one step further. In Python we can define functions inside other functions:

```
def hi(name="yasoob"):
    print("now you are inside the hi() function")

    def greet():
        return "now you are in the greet() function"

    def welcome():
        return "now you are in the welcome() function"

    print(greet())
    print(welcome())
    print("now you are back in the hi() function")

hi()
#output:now you are inside the hi() function
#       now you are in the greet() function
#       now you are in the welcome() function
#       now you are back in the hi() function

# This shows that whenever you call hi(), greet() and welcome()
# are also called. However the greet() and welcome() functions
# are not available outside the hi() function e.g:

greet()
#outputs: NameError: name 'greet' is not defined
```

Returning functions from within functions:

It is not necessary to execute a function within another function, we can return it as an output as well:

```
def hi(name="yasoob"):
    def greet():
        return "now you are in the greet() function"

    def welcome():
        return "now you are in the welcome() function"

    if name == "yasoob":
        return greet
    else:
        return welcome

a = hi()
print(a)
#outputs: <function greet at 0x7f2143c01500>

#This clearly shows that `a` now points to the greet() function in hi()
#Now try this

print(a())
#outputs: now you are in the greet() function
```

Returning functions from within functions:

Just take a look at the code again. In the if/else clause we are returning greet and welcome, not greet() and welcome(). Why is that? It’s because when you put a pair of parentheses after it, the function gets executed; whereas if you don’t put parenthesis after it, then it can be passed around and can be assigned to other variables without executing it. Did you get it? Let me explain it in a little bit more detail. When we write a = hi(), hi() gets executed and because the name is yasoob by default, the function greet is returned. If we change the statement to a = hi(name = "ali") then the welcome function will be returned. We can also do print hi()() which outputs now you are in the greet() function.

Giving a function as an argument to another function:

```
def hi():
    return "hi yasoob!"

def doSomethingBeforeHi(func):
    print("I am doing some boring work before executing hi()")
    print(func())

doSomethingBeforeHi(hi)
#outputs:I am doing some boring work before executing hi()
#        hi yasoob!
```

Now you have all the required knowledge to learn what decorators really are. Decorators let you execute code before and after a function.

Writing your first decorator:

In the last example we actually made a decorator! Let's modify the previous decorator and make a little bit more usable program:

```
def a_new_decorator(a_func):

    def wrapTheFunction():
        print("I am doing some boring work before executing a_func()")

        a_func()

        print("I am doing some boring work after executing a_func()")

    return wrapTheFunction

def a_function_requiring_decoration():
    print("I am the function which needs some decoration to remove my foul smell")

a_function_requiring_decoration()
#outputs: "I am the function which needs some decoration to remove my foul smell"

a_function_requiring_decoration = a_new_decorator(a_function_requiring_decoration)
#now a_function_requiring_decoration is wrapped by wrapTheFunction()

a_function_requiring_decoration()
#outputs:I am doing some boring work before executing a_func()
#        I am the function which needs some decoration to remove my foul smell
#        I am doing some boring work after executing a_func()
```

Writing your first decorator:

Did you get it?

We just applied the previously learned principles. This is exactly what the decorators do in Python! They wrap a function and modify its behaviour in one way or another. Now you might be wondering why we did not use the @ anywhere in our code? That is just a short way of making up a decorated function. Here is how we could have run the previous code sample using @

```
@a_new_decorator
def a_function_requiring_decoration():
    """Hey you! Decorate me!"""
    print("I am the function which needs some decoration to "
          "remove my foul smell")

a_function_requiring_decoration()
#outputs: I am doing some boring work before executing a_func()
#         I am the function which needs some decoration to remove my foul smell
#         I am doing some boring work after executing a_func()

#the @a_new_decorator is just a short way of saying:
a_function_requiring_decoration = a_new_decorator(a_function_requiring_decoration)
```

Writing your first decorator

I hope you now have a basic understanding of how decorators work in Python. Now there is one problem with our code. If we run:

```
print(a_function_requiring_decoration.__name__)
# Output: wrapTheFunction
```

Writing your first decorator

That’s not what we expected! Its name is “a_function_requiring_decoration”. Well, our function was replaced by wrapTheFunction. It overrode the name and docstring of our function. Luckily, Python provides us a simple function to solve this problem and that is functools.wraps. Let’s modify our previous example to use functools.wraps:

```
from functools import wraps

def a_new_decorator(a_func):
    @wraps(a_func)
    def wrapTheFunction():
        print("I am doing some boring work before executing a_func()")
        a_func()
        print("I am doing some boring work after executing a_func()")
    return wrapTheFunction

@a_new_decorator
def a_function_requiring_decoration():
    """Hey yo! Decorate me!"""
    print("I am the function which needs some decoration to "
          "remove my foul smell")

print(a_function_requiring_decoration.__name__)
# Output: a_function_requiring_decoration
```

Now that is much better. Let’s move on and learn some use-cases of decorators.

Blueprint

```
from functools import wraps
def decorator_name(f):
    @wraps(f)
    def decorated(*args, **kwargs):
        if not can_run:
            return "Function will not run"
        return f(*args, **kwargs)
    return decorated

@decorator_name
def func():
    return("Function is running")

can_run = True
print(func())
# Output: Function is running

can_run = False
print(func())
# Output: Function will not run
```

Note: @wraps takes a function to be decorated and adds the functionality of copying over the function name, docstring, arguments list, etc. This allows us to access the pre-decorated function's properties in the decorator.

Use-cases

Now let’s take a look at the areas where decorators really shine and their usage makes something really easy to manage.

Authorization

Decorators can help to check whether someone is authorized to use an endpoint in a web application. They are extensively used in Flask web framework and Django. Here is an example to employ decorator based authentication

Example :

```
from functools import wraps

def requires_auth(f):
    @wraps(f)
    def decorated(*args, **kwargs):
        auth = request.authorization
        if not auth or not check_auth(auth.username, auth.password):
            authenticate()
        return f(*args, **kwargs)
    return decorated
```

Logging

Logging is another area where the decorators shine. Here is an example:

```
from functools import wraps

def logit(func):
    @wraps(func)
    def with_logging(*args, **kwargs):
        print(func.__name__ + " was called")
        return func(*args, **kwargs)
    return with_logging

@logit
def addition_func(x):
   """Do some math."""
   return x + x


result = addition_func(4)
# Output: addition_func was called
```

I am sure you are already thinking about some clever uses of decorators.

Decorators with Arguments

Come to think of it, isn't @wraps also a decorator? But, it takes an argument like any normal function can do. So, why can't we do that too?

This is because when you use the @my_decorator syntax, you are applying a wrapper function with a single function as a parameter. Remember, everything in Python is an object, and this includes functions! With that in mind, we can write a function that returns a wrapper function.

```python
from functools import wraps

def logit(logfile='out.log'):
    def logging_decorator(func):
        @wraps(func)
        def wrapped_function(*args, **kwargs):
            log_string = func.__name__ + " was called"
            print(log_string)
            # Open the logfile and append
            with open(logfile, 'a') as opened_file:
                # Now we log to the specified logfile
                opened_file.write(log_string + '\n')
            return func(*args, **kwargs)
        return wrapped_function
    return logging_decorator

@logit()
def myfunc1():
    pass

myfunc1()
# Output: myfunc1 was called
# A file called out.log now exists, with the above string

@logit(logfile='func2.log')
def myfunc2():
    pass

myfunc2()
# Output: myfunc2 was called
# A file called func2.log now exists, with the above string
```

Nesting a Decorator Within a Function

Let’s go back to our logging example, and create a wrapper which lets us specify a logfile to output to.

Decorator Classes

Now we have our logit decorator in production, but when some parts of our application are considered critical, failure might be something that needs more immediate attention. Let’s say sometimes you want to just log to a file. Other times you want an email sent, so the problem is brought to your attention, and still keep a log for your own records. This is a case for using inheritence, but so far we’ve only seen functions being used to build decorators.

Decorator Classes

Luckily, classes can also be used to build decorators. So, let's rebuild logit as a class instead of a function.

```
class logit(object):

    _logfile = 'out.log'

    def __init__(self, func):
        self.func = func

    def __call__(self, *args):
        log_string = self.func.__name__ + " was called"
        print(log_string)
        # Open the logfile and append
        with open(self._logfile, 'a') as opened_file:
            # Now we log to the specified logfile
            opened_file.write(log_string + '\n')
        # Now, send a notification
        self.notify()

        # return base func
        return self.func(*args)



    def notify(self):
        # logit only logs, no more
        pass
```

Decorator Classes

This implementation has an additional advantage of being much cleaner than the nested function approach, and wrapping a function still will use the same syntax as before:

```
logit._logfile = 'out2.log' # if change log file
@logit
def myfunc1():
    pass

myfunc1()
# Output: myfunc1 was called
```

Decorator Classes

Now, let's subclass logit to add email functionality (though this topic will not be covered here).

```
class email_logit(logit):
    '''
    A logit implementation for sending emails to admins
    when the function is called.
    '''
    def __init__(self, email='admin@myproject.com', *args, **kwargs):
        self.email = email
        super(email_logit, self).__init__(*args, **kwargs)

    def notify(self):
        # Send an email to self.email
        # Will not be implemented here
        pass
```

From here, @email_logit works just like @logit but sends an email to the admin in addition to logging.

Why you need to learn Python?

- 1. Performance tuning
- 2. Cross Language Support (Inter-Ops)
- 3. Clean Code
- 4. Interview

But have you ever think of

What actually is Python?

Compiled vs. Interpreted Languages

- Compiled language - a programming language in which one writes the code, and then “compiles” it into a separate file (program) that the computer can understand, which then gets executed.

  Examples: C, C++, Java, Rust, Fortran

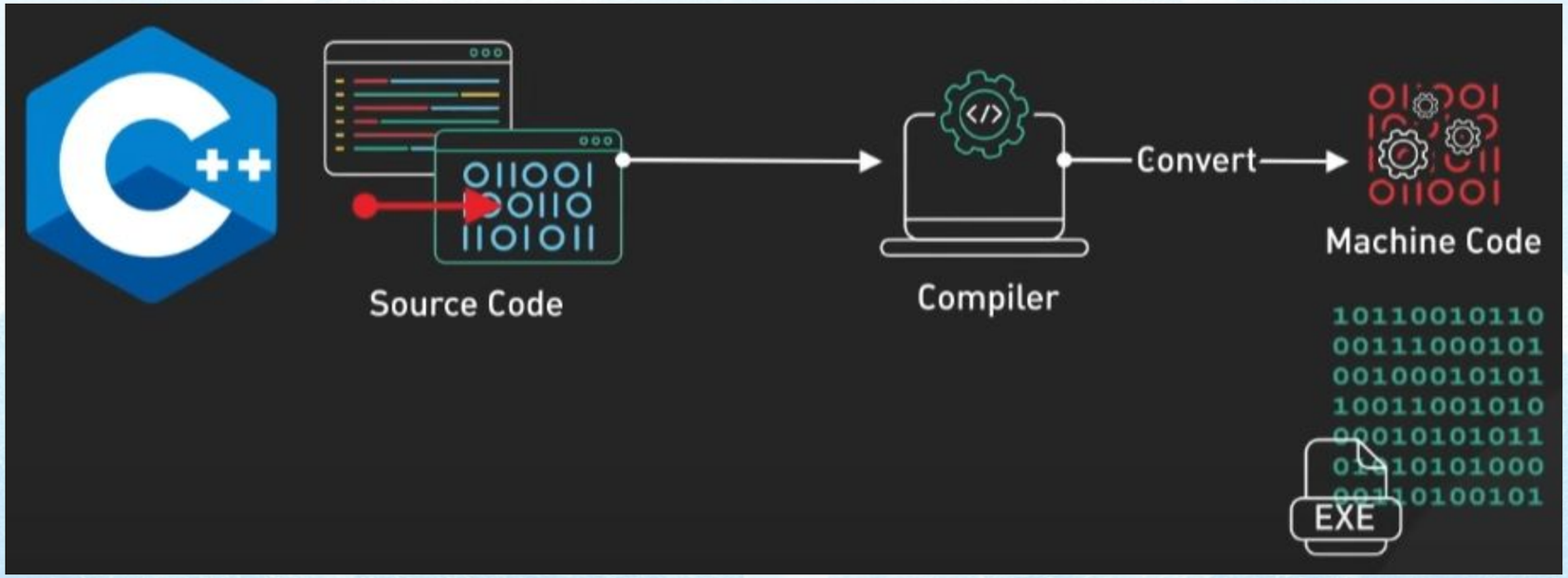
- Interpreted language - a programming language in which one writes the code, and that code gets “interpreted” into what the computer can understand at runtime. No separate file is created that one has to execute, like with a compiled language.

  Examples: Python, Bash, Javascript, Ruby, Perl

- The trade-off is that generally a compiled language could be more difficult to learn, slower to write, but will execute faster, and vice versa.

Programming Languages

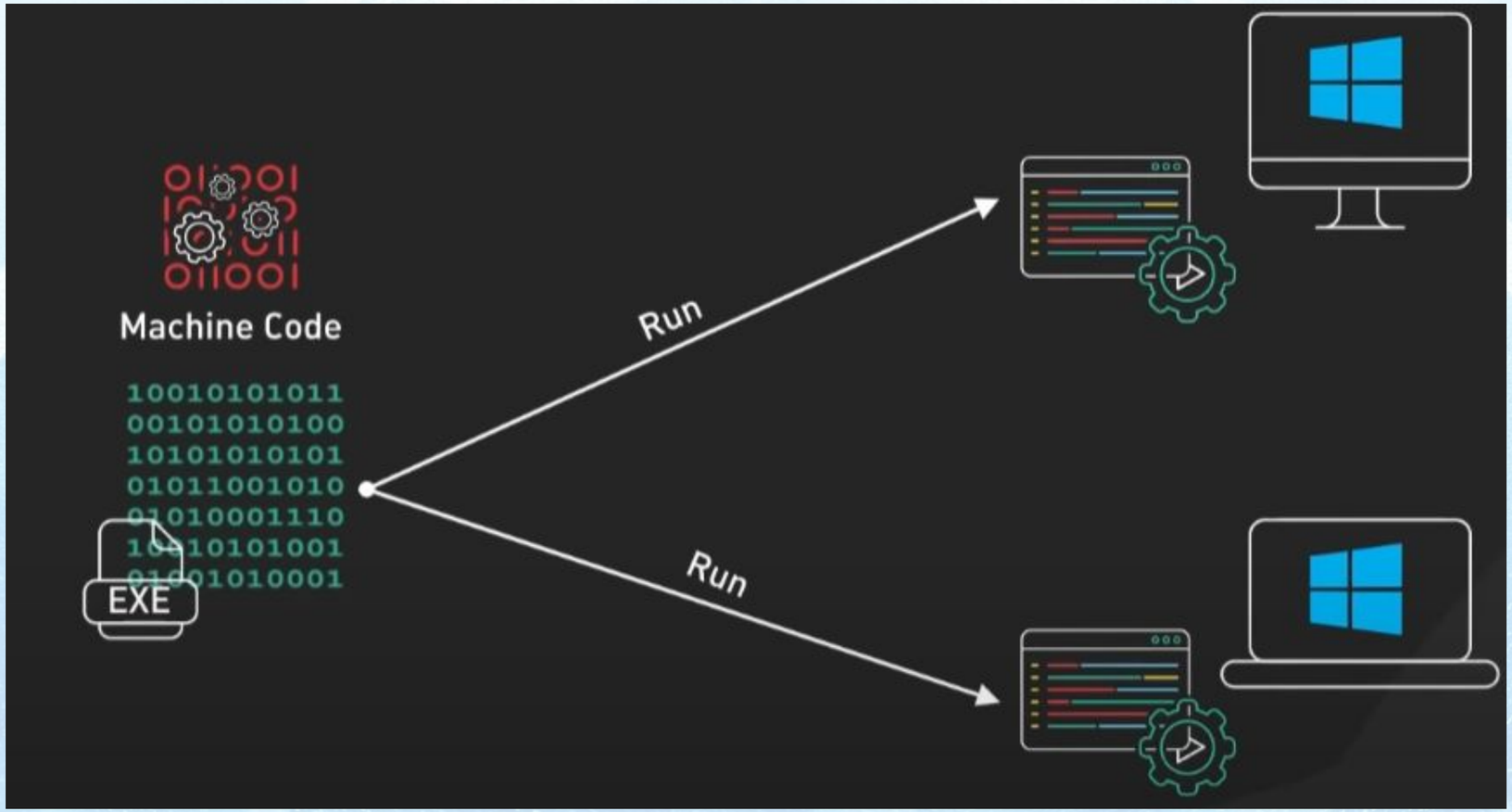
- Python Vs C++ Vs Java - C++



- C++ is a compiled language.
- It begins its journey when the source code is sent to a Compiler.
- The Compiler meticulously analyzes the code and translates it into machine code - simple CPU instructions like add, move, and jump.
- The Compiler outputs an Executable File containing the machine code.

Programming Languages

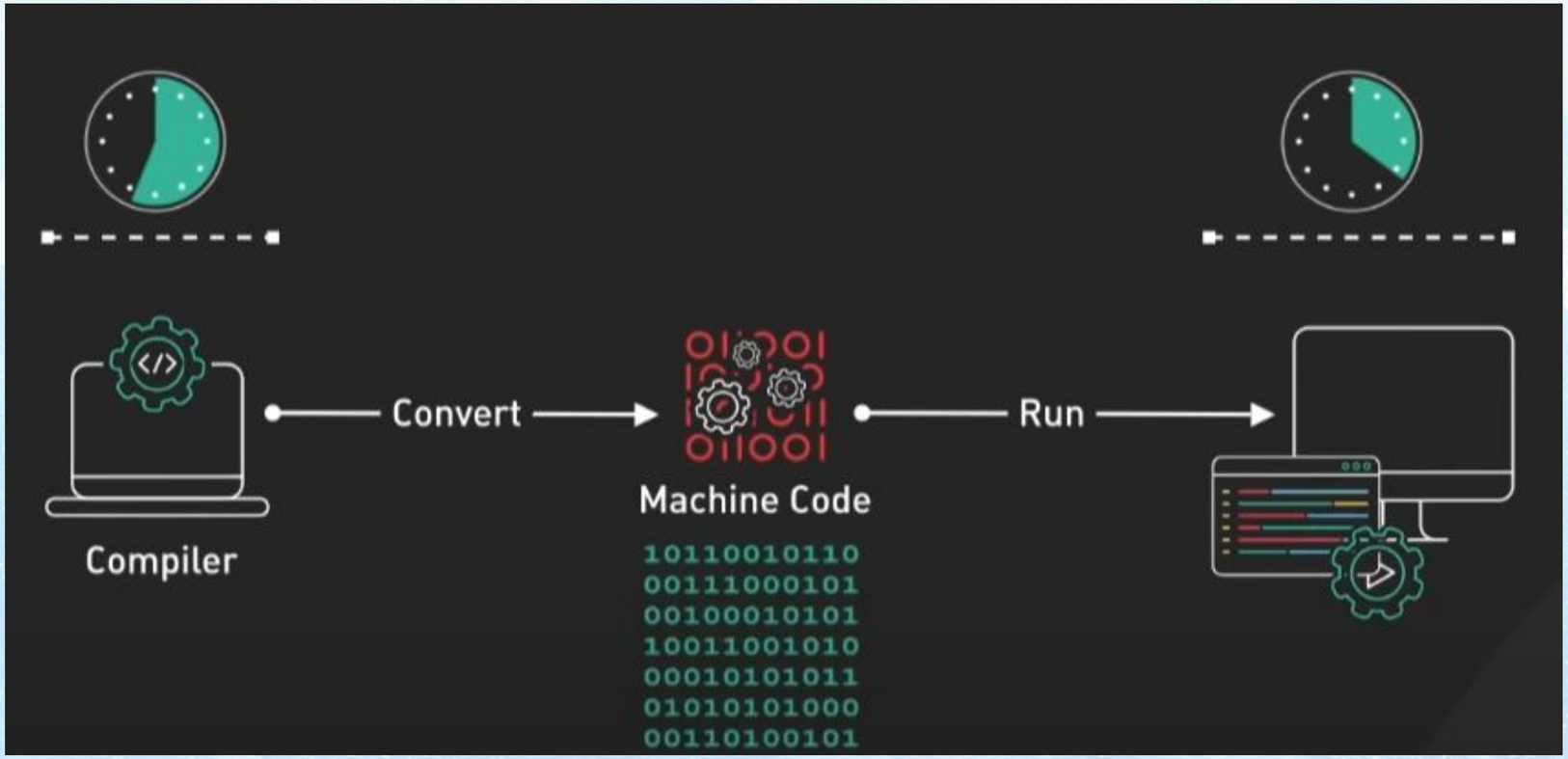
- Python Vs C++ Vs Java - C++



- This file is a standalone program that can be run on any compatible computer.
- This compiled nature makes C++ a great performer.
- It’s often chosen for performance-sensitive tasks, such as gaming or system-level programming.

Programming Languages

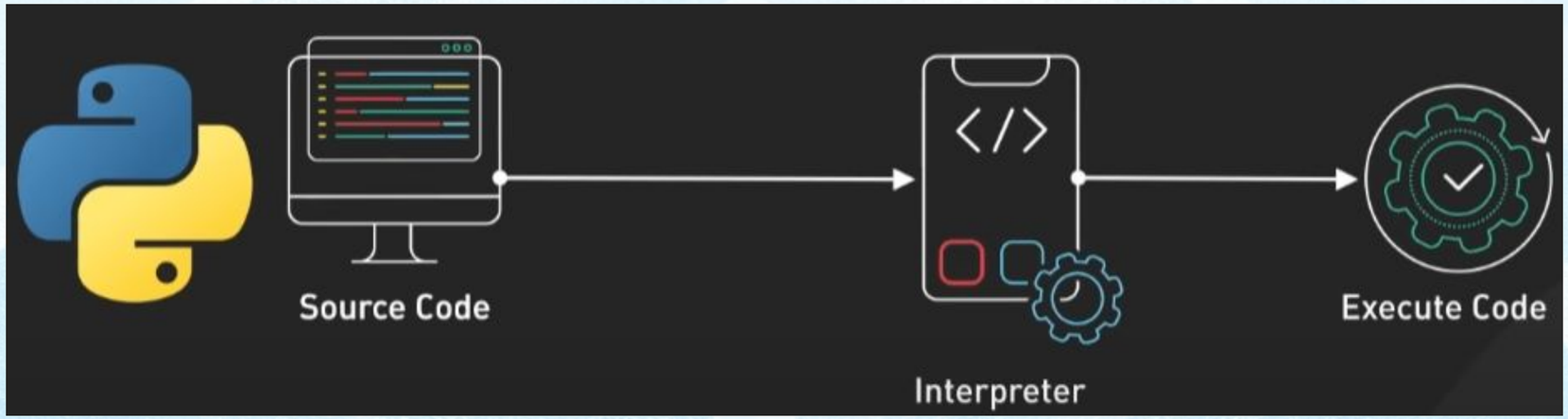
- Python Vs C++ Vs Java - C++



- Compiled languages like C++, Go, and Rust take longer to compile initially, but then run very fast as the CPU doesn't need to interpret or Just-In-Time compile the code.

Programming Languages

- Python Vs C++ Vs Java - Python



- Now let's look at Python, an interpreted language. It takes a different approach.
- Instead of compiling, Python sends its source code straight to the Interpreter.
- This reads and executes the code on-the-fly.
- This process makes Python exceptionally flexible and user-friendly.
- It is ideal for situations where rapid development and readability are important.

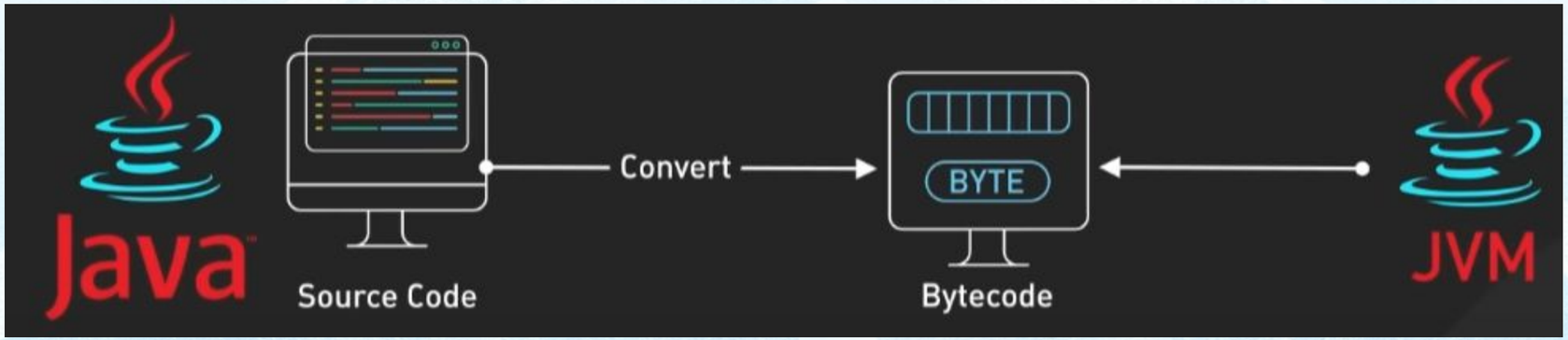
Programming Languages

- Python Vs C++ Vs Java - Python

- However, this also means it’s generally slower than compiled languages, as each line must be interpreted every time it's executed.
- Other popular interpreted languages include Javascript, Ruby, and Perl.

Programming Languages

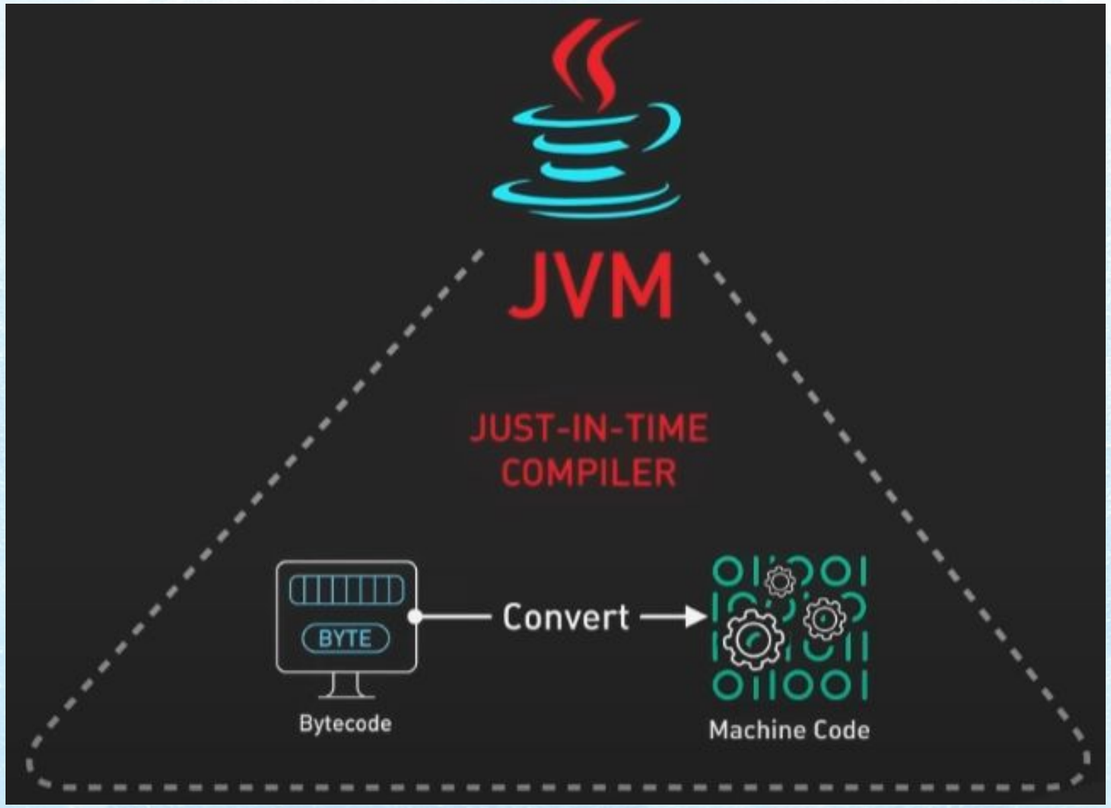
- Python Vs C++ Vs Java - Java



- Then there’s Java. Java uses a unique, hybrid approach.
- First, Java source code gets compiled into bytecode, which is then executed by the Java Virtual Machine (JVM).

Programming Languages

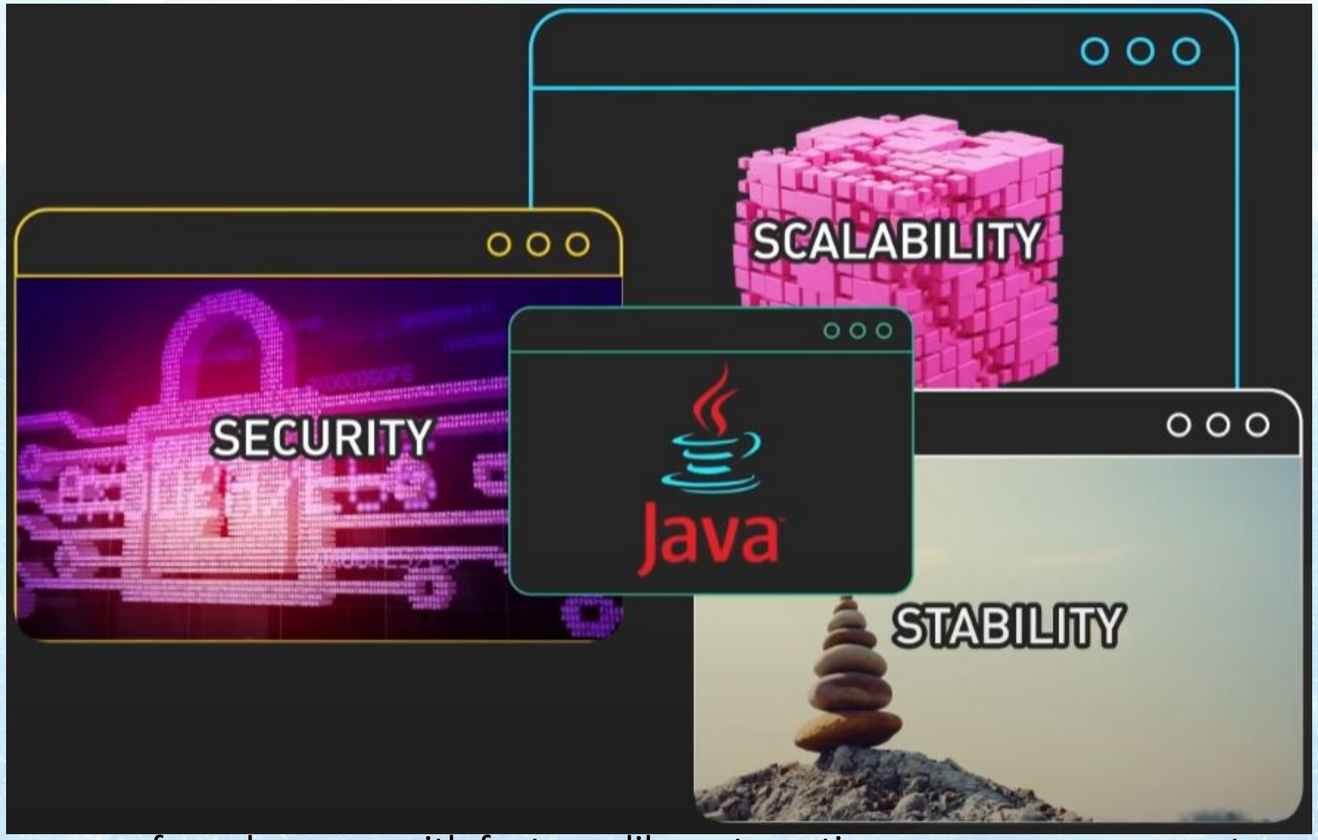
- Python Vs C++ Vs Java - Java



- Here’s the secret sauce – the JVM has a Just-In-Time compiler that dynamically converts performance-critical bytecode into optimized native machine code right before execution.

Programming Languages

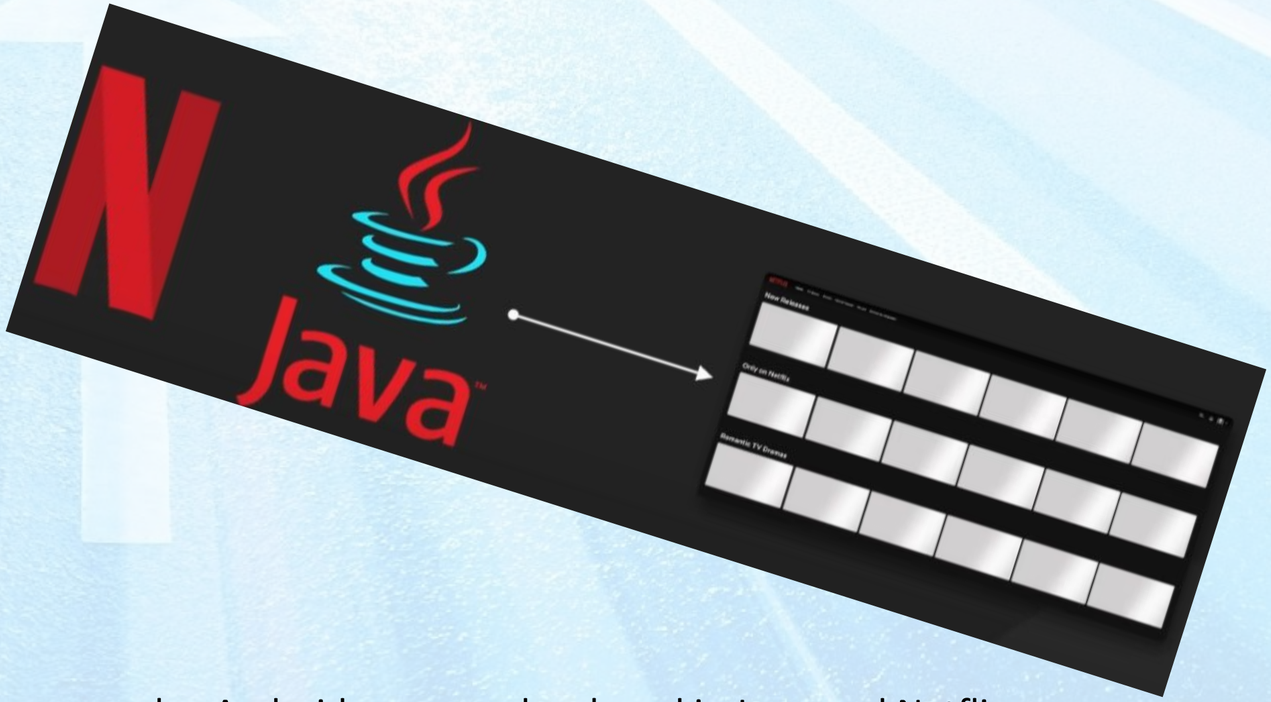
- Python Vs C++ Vs Java - Java



- Java is also designed to be memory safe and secure, with features like automatic memory management.
- This makes it well-suited for large, complex enterprise applications that demand stability, security and scalability.

Programming Languages

- Python Vs C++ Vs Java - Java



- Leading tech companies use Java to power critical systems - for example, Android apps are developed in Java, and Netflix uses Java across its infrastructure.

Programming Languages

- Python Vs C++ Vs Java - Summary
  - Other bytecode language examples are C# and Kotlin.

  - Thanks to advances like Just-In-Time compilation, the lines between interpreted and bytecode languages are blurring.

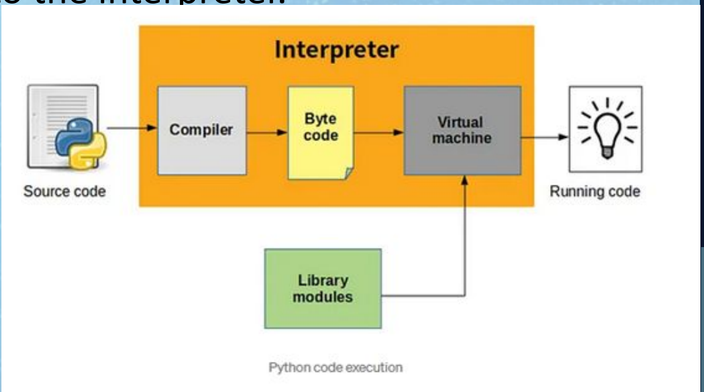
Programming Languages

- Python Vs C++ Vs Java - Summary
  - Modern JavaScript engines use JIT to optimize performance. However, overall JavaScript still falls into the interpreted category.

- C++ offers raw performance, Python flexibility, and Java balances both. Understanding these core differences under the hood explains why these languages excel in different situations.

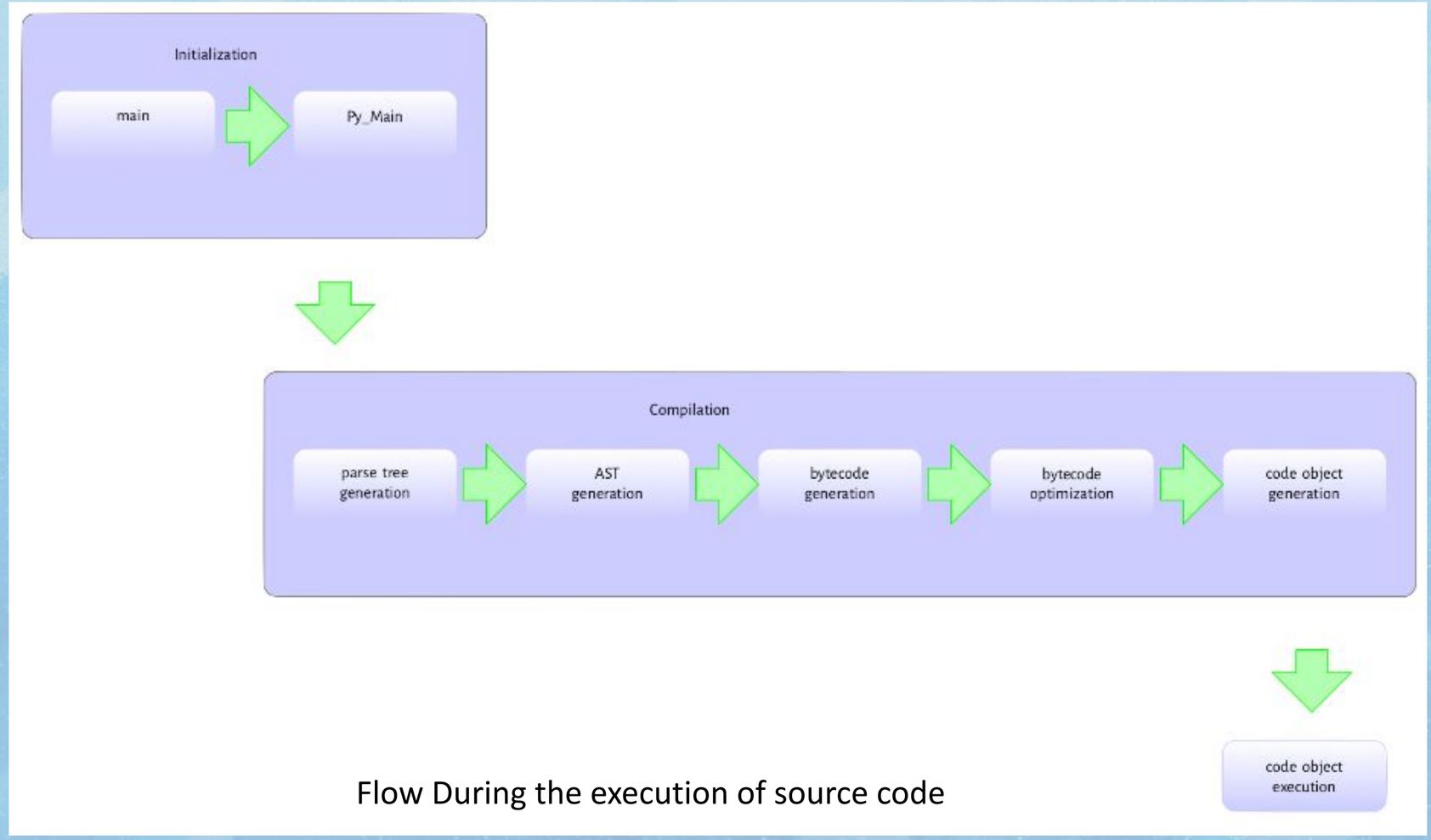
Introduction to Python

- - Python, developed by Guido van Rossum in 1989, has become a popular programming language used in various applications.
- - The focus of the write-up is on the Python interpreter, specifically CPython, the reference implementation.
- - The execution of a Python program can be divided into initialization, compiling, and interpreting phases.
- - Initialization involves setting up data structures when a program is executed non-interactively.
- - Compiling includes tasks such as building syntax trees, creating abstract syntax trees, and generating code objects.
- - Interpreting involves executing bytecode within a specific context.
- - The write-up pays particular attention to building symbol tables and code objects from the abstract syntax tree.
- - The material assumes familiarity with Python and a basic understanding of the C programming language.
- - It provides insights into the processes and data structures essential to executing a Python program.
- - The author acknowledges the influence of various sources and recommends exploring Pycon videos, school lectures, and blog write-ups.
- - The write-up starts with an overview of executing a script passed as a command-line argument to the interpreter.
- - It is intended for individuals interested in understanding the inner workings of the CPython virtual machine.



The View From 30,000ft

- Python executable is a C program like any other C program such as the Linux kernel or a simple hello world program in C



Flow During the execution of source code

Compiling Python Source Code

- Disassembling a python function

```
>>> from dis import dis
        >>> def square(x):
        ...     return x*x
        ...

        >>> dis(square)
        2           0 LOAD_FAST                0 (x)
                    2 LOAD_FAST                0 (x)
                    4 BINARY_MULTIPLY
                    6 RETURN_VALUE
```

Compiling Python Source Code

- 1. Parsing the source code into a parse tree.
- 2. Transforming the parse tree into an abstract syntax tree (AST).
- 3. Generating the symbol table.
- 4. Generating the code object from the AST. This step involves:

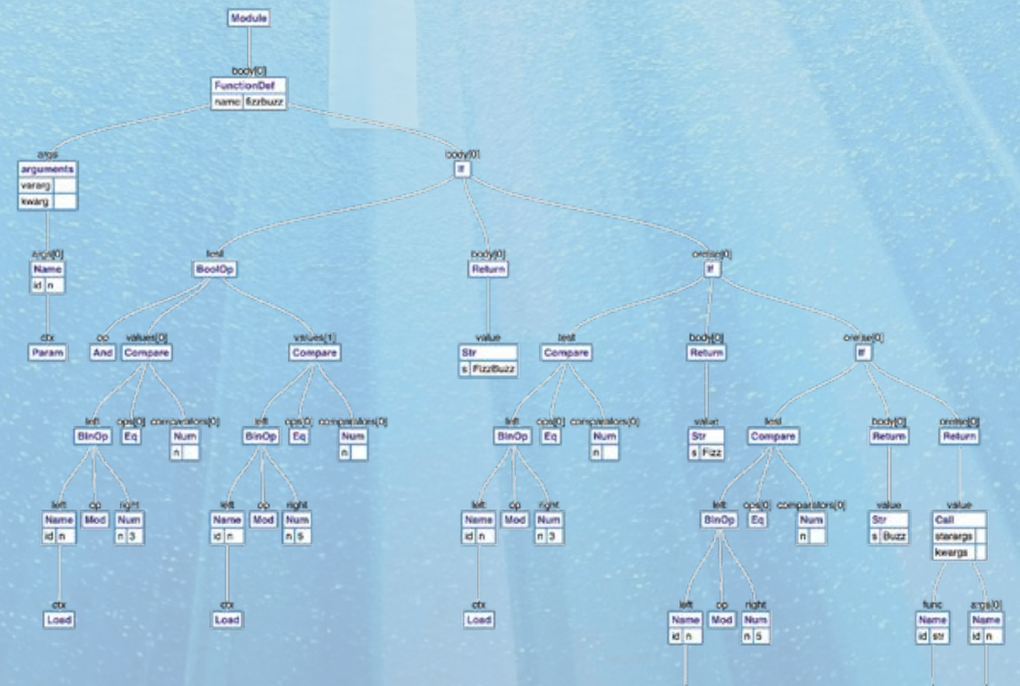
    1. Transforming the AST into a flow control graph, and
    2. Emitting a code object from the control flow graph

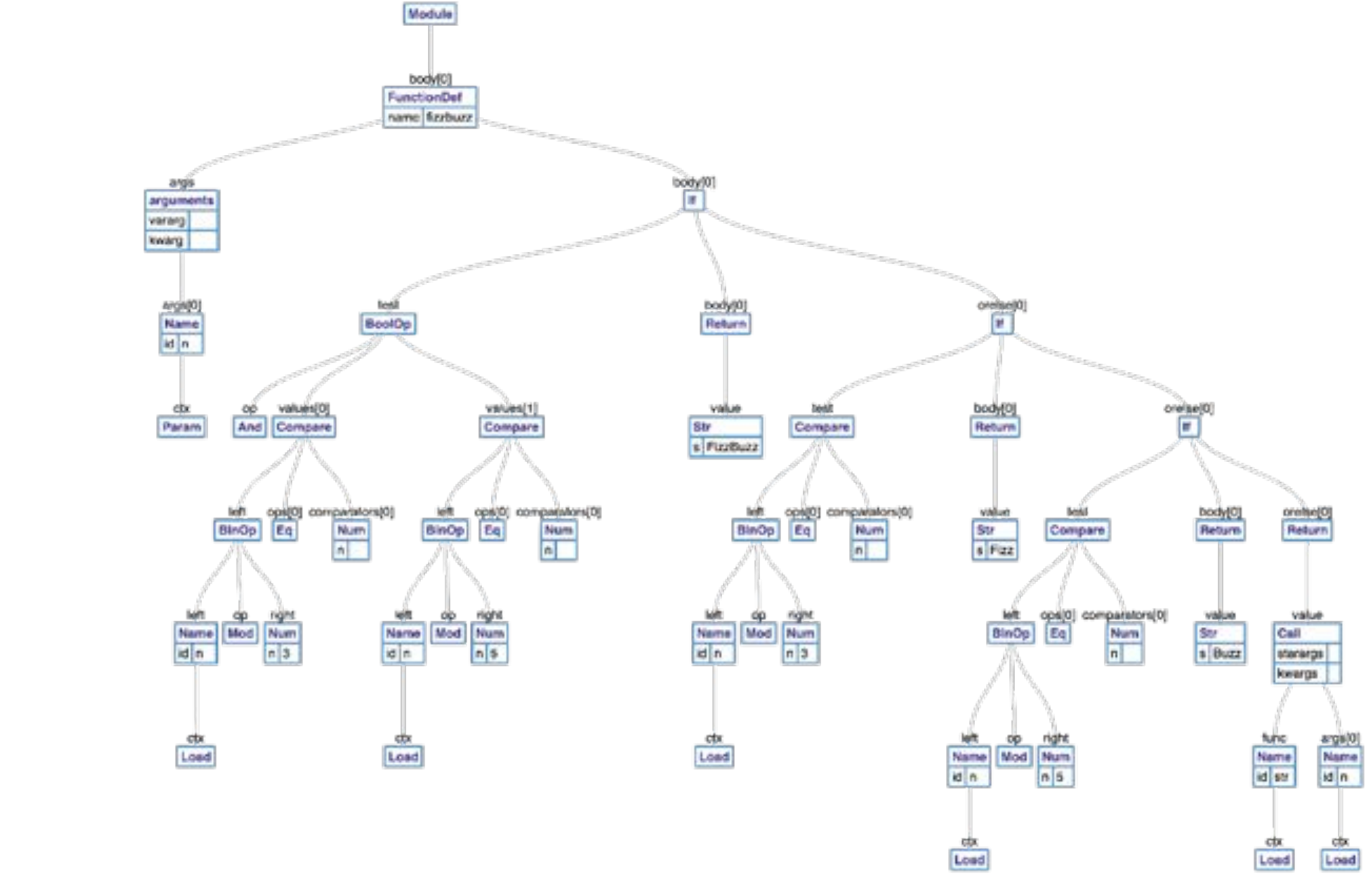
Mental Picture of the Process

- Python Program - A simple python function

```
1 def fizzbuzz(n):
2     if n % 3 == 0 and n % 5 == 0:
3         return 'FizzBuzz'
4     elif n % 3 == 0:
5         return 'Fizz'
6     elif n % 5 == 0:
7         return 'Buzz'
8     else:
9         return str(n)
```

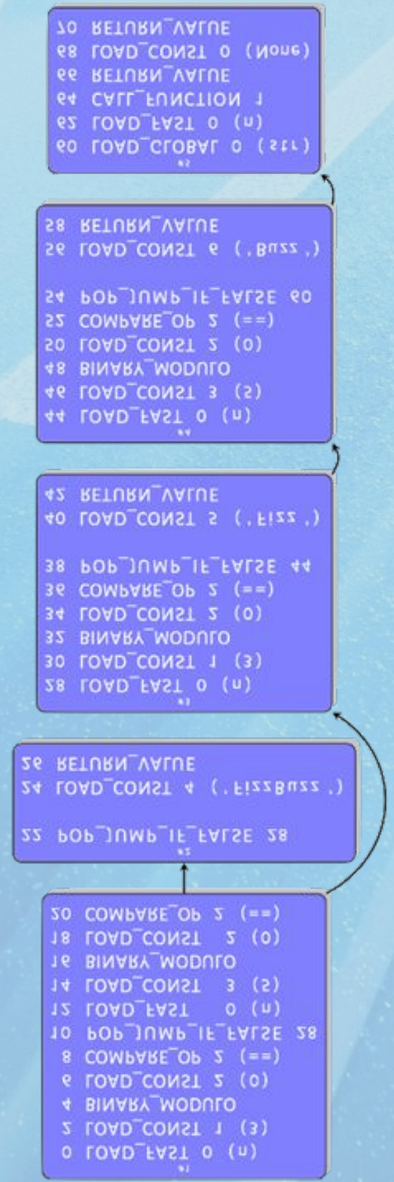
- Abstract Syntax Tree (AST)




Module
body[0]
FunctionDef
name fizzbuzz
args
arguments
vararg
kwarg
body[0]
If
args[0]
Name
id n
ctx
Param
test
BoolOp
body[0]
Return
orelse[0]
If
op
And
values[0]
Compare
values[1]
Compare
value
Str
s FizzBuzz
test
Compare
body[0]
Return
orelse[0]
If
left
BinOp
ops[0]
Eq
comparators[0]
Num
n
left
BinOp
ops[0]
Eq
comparators[0]
Num
n
left
BinOp
ops[0]
Eq
comparators[0]
Num
n
value
Str
s Fizz
test
Compare
body[0]
Return
orelse[0]
If
left
Name
id n
op
Mod
right
Num
n 3
left
Name
id n
op
Mod
right
Num
n 5
left
Name
id n
op
Mod
right
Num
n 3
left
BinOp
ops[0]
Eq
comparators[0]
Num
n
value
Str
s Buzz
value
Call
starargs
kwargs
ctx
Load
ctx
Load
ctx
Load
ctx
Load
ctx
Load
func
Name
id str
args[0]
Name
id n
ctx
Load
ctx
Load

Mental Picture of the Process

- Compiled Byte Code



- Code Object

```c
typedef struct basicblock_ {
    /* Each basicblock in a compilation unit is linked via b_list in the
    reverse order that the block are allocated.  b_list points to the next
    block, not to be confused with b_next, which is next by control flow. */
    struct basicblock_ *b_list;
    /* number of instructions used */
    int b_iused;
    /* length of instruction array (b_instr) */
    int b_ialloc;
    /* pointer to an array of instructions, initially NULL */
    struct instr *b_instr;
    /* If b_next is non-NULL, it is a pointer to the next
    block reached by normal control flow. */
    struct basicblock_ *b_next;
    /* b_seen is used to perform a DFS of basicblocks. */
    unsigned b_seen : 1;
    /* b_return is true if a RETURN_VALUE opcode is inserted. */
    unsigned b_return : 1;
    /* depth of stack upon entry of block, computed by stackdepth() */
    int b_startdepth;
    /* instruction offset for block, computed by assemble_jump_offsets() */
    int b_offset;
} basicblock;
```

Python Virtual Machine

- Simple hello world python function

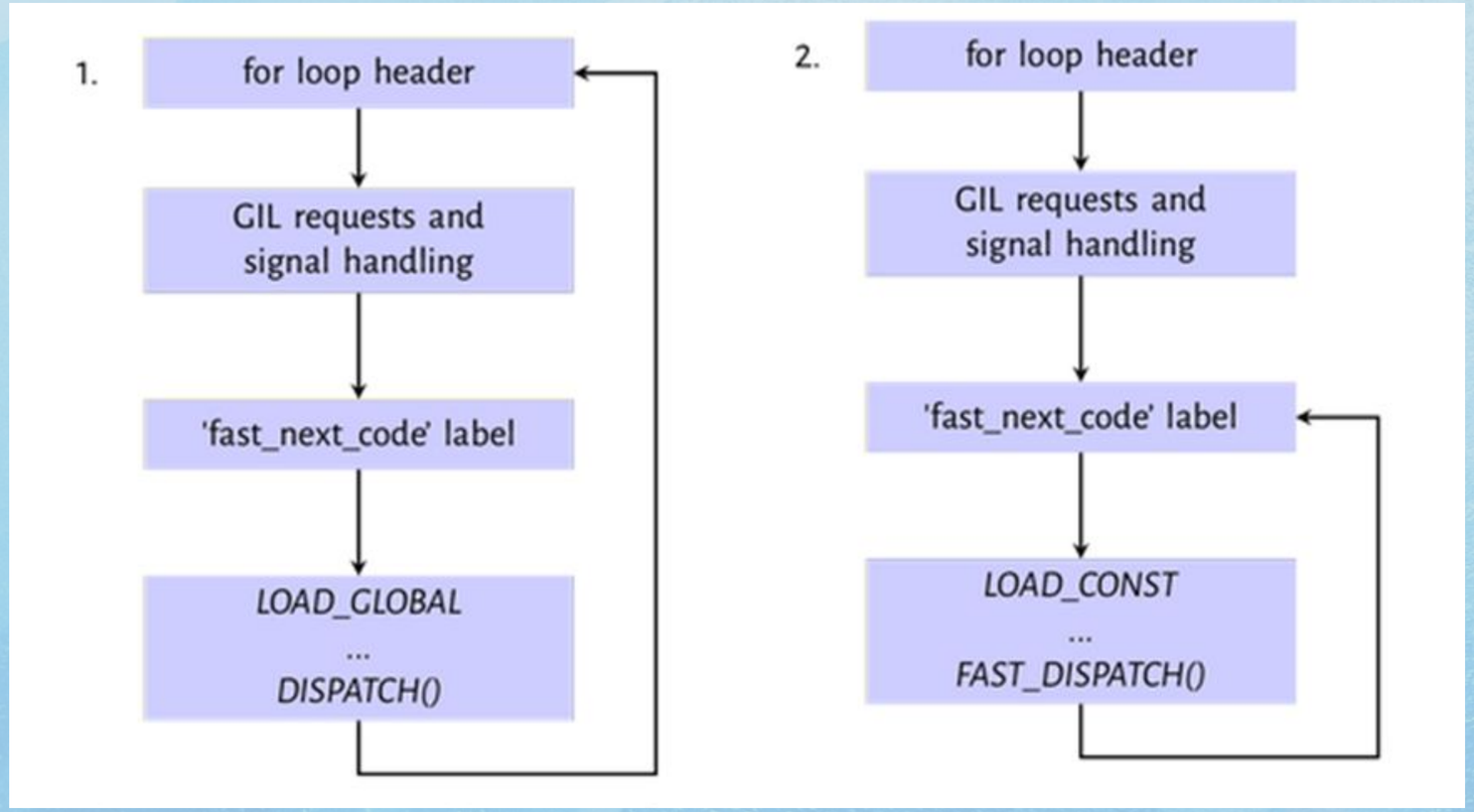
```
def hello_world():
    print("hello world")
```

- Disassembly of the above function

```
LOAD_GLOBAL              0 (0)
LOAD_CONST               1 (1)
CALL_FUNCTION            1 (1 positional, 0 keyword pair)
POP_TOP
LOAD_CONST               0 (0)
RETURN_VALUE
```

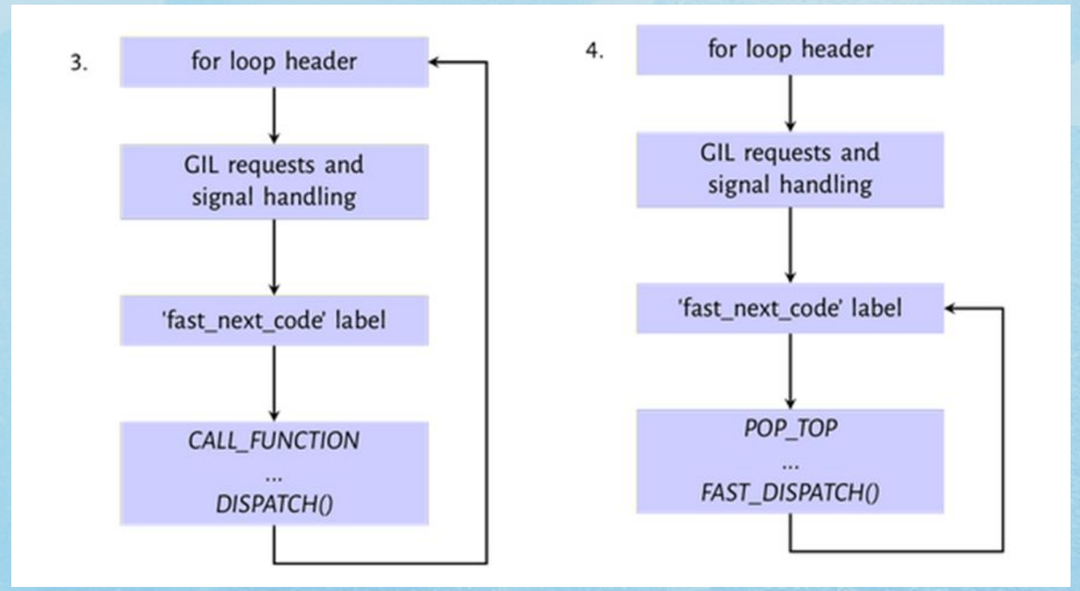
Python Virtual Machine

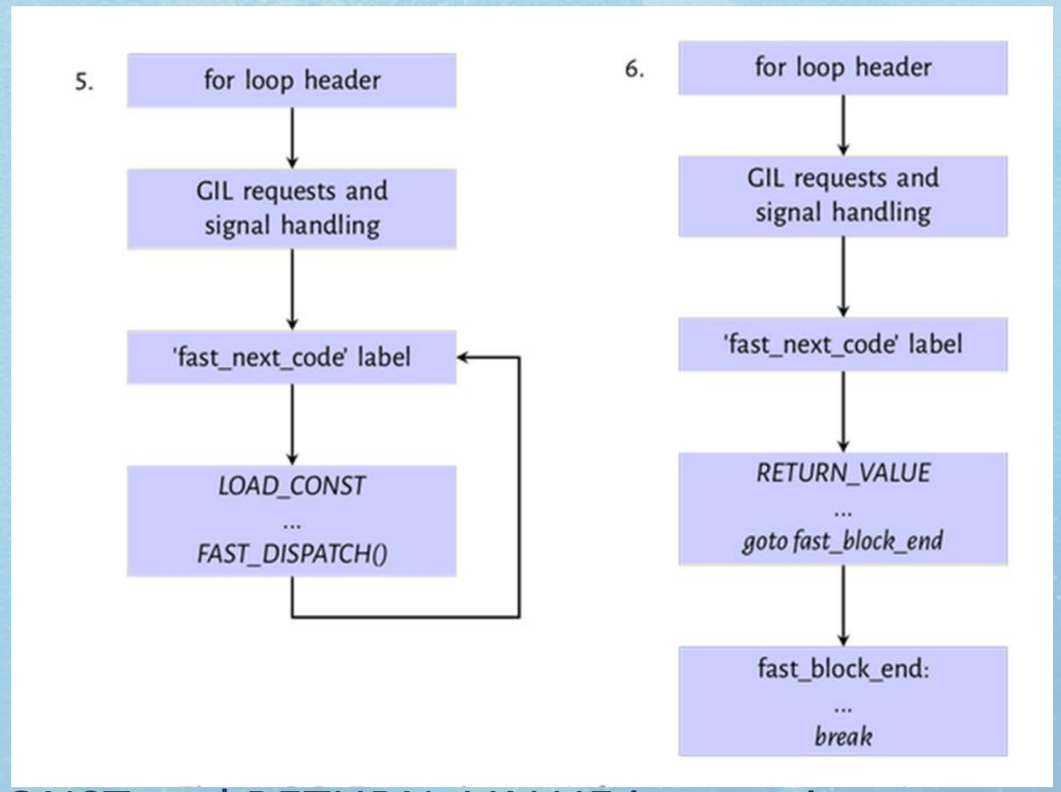
- Evaluation path for LOAD_GLOBAL and LOAD_CONST instructions



Python Virtual Machine

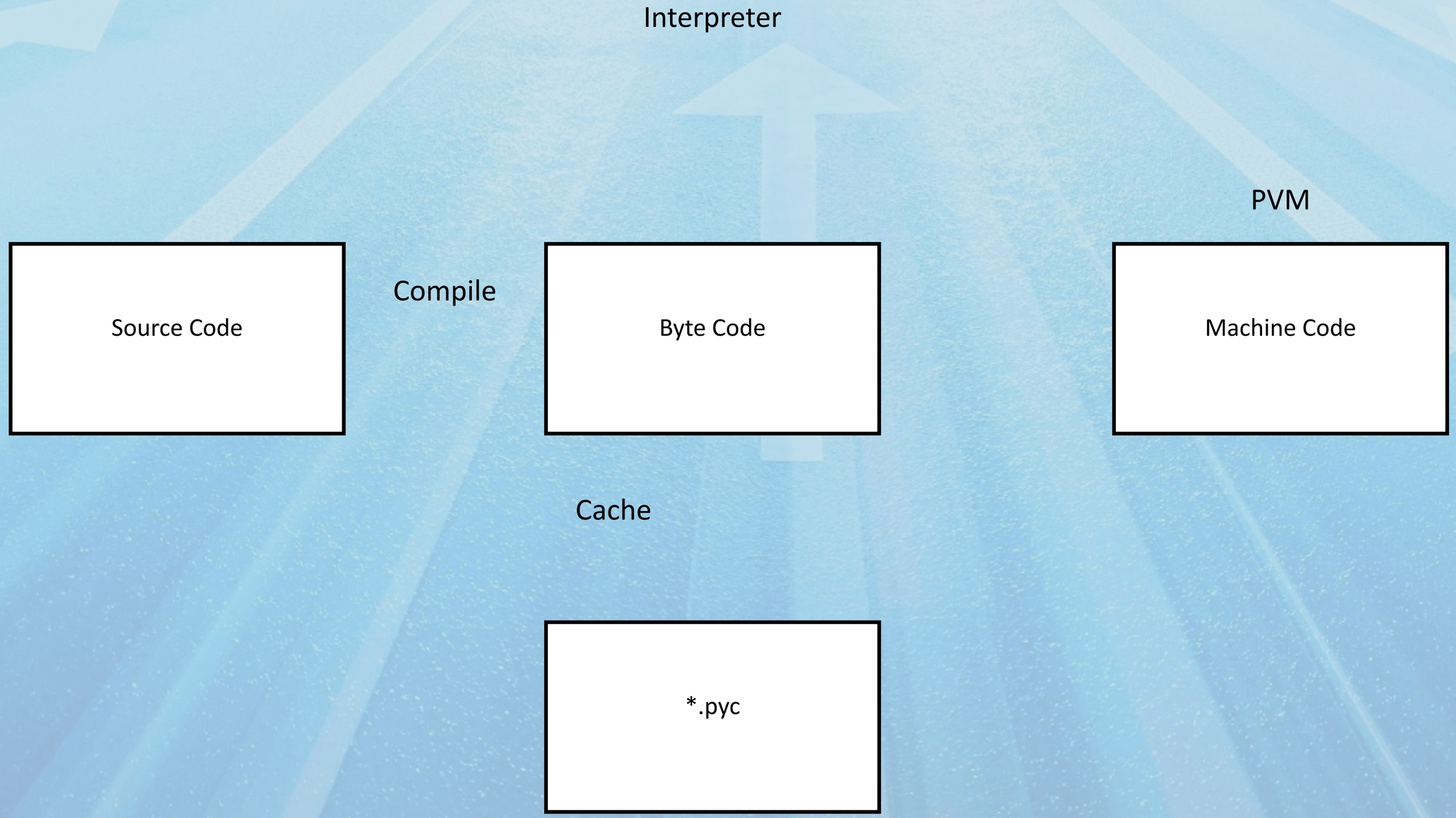
- Evaluation path for CALL_FUNCTION and POP_TOP insturction





- Evaluation path for LOAD_CONST and RETURN_VALUE instruction

Summary


Interpreter
PVM
Source Code
Compile
Byte Code
Machine Code
Cache
*.pyc

Cpython, GIL and Performance

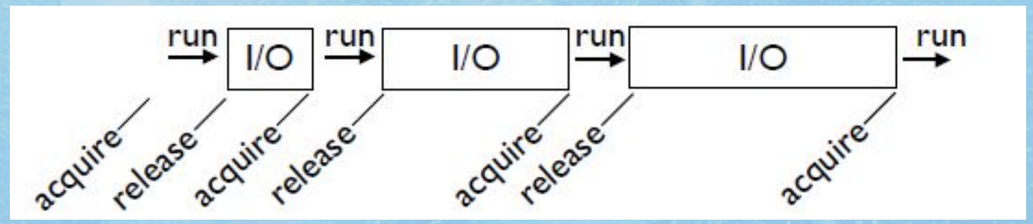
- The Infamous GIL
- Here's the rub...

- Only one Python thread can execute in the interpreter at once

- There is a "global interpreter lock" that carefully controls thread execution

- The GIL ensures that sure each thread gets exclusive access to the interpreter internals when it's running (and that call-outs to C extensions play nice)

- GIL Behavior
- It's simple : threads hold the GIL when running



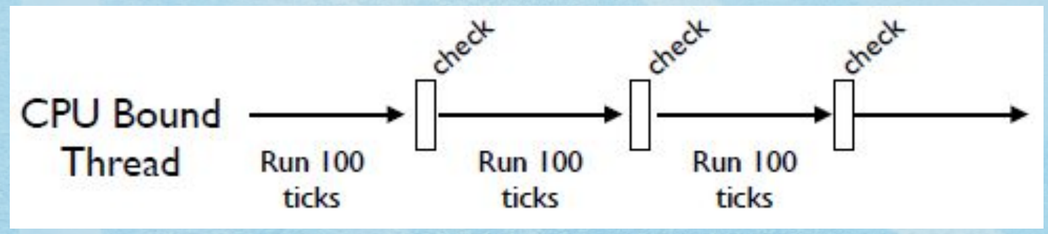
- However, they release it when blocking for I/O

- So, any time a thread is forced to wait, other "ready" threads get their chance to run

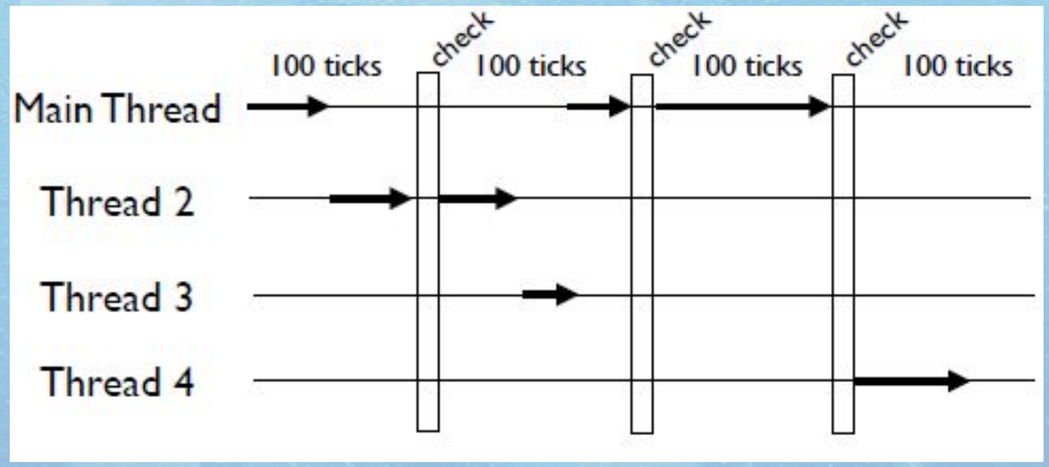
- Basically a kind of "cooperative" multitasking

Cpython, GIL and Performance

- CPU Bound Processing
  - To deal with CPU-bound threads that never perform any I/O, the interpreter periodically performs a "check"
  - By default, every 100 interpreter "ticks"
  - sys.setcheckinterval() changes the setting



- The Check Interval
  - The check interval is a global counter that is completely independent of thread scheduling
  - A "check" is simply made every 100 "ticks"



Cpython, GIL and Performance

The Periodic Check

- What happens during the periodic check? In the main thread only, signal handlers will execute if there are any pending signals (more shortly)

- Release and reacquire the GIL

- That last bullet describes how multiple CPUbound threads get to run (by briefly releasing the GIL, other threads get a chance to run).

GIL Implementation

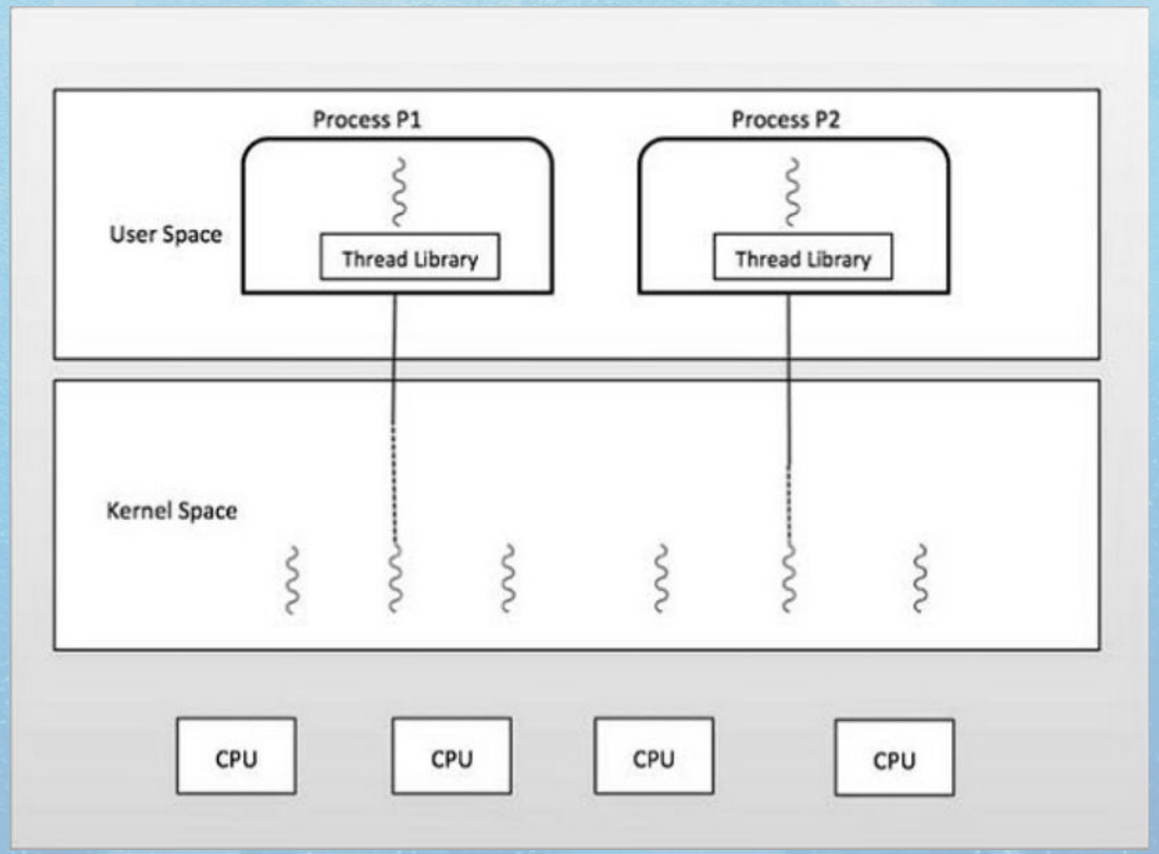
- The GIL is not a simple mutex lock
- The implementation (Unix) is either...

- A POSIX unnamed semaphore
- Or a pthreads condition variable

- All interpreter locking is based on signaling
- To acquire the GIL, check if it's free. If not, go to sleep and wait for a signal

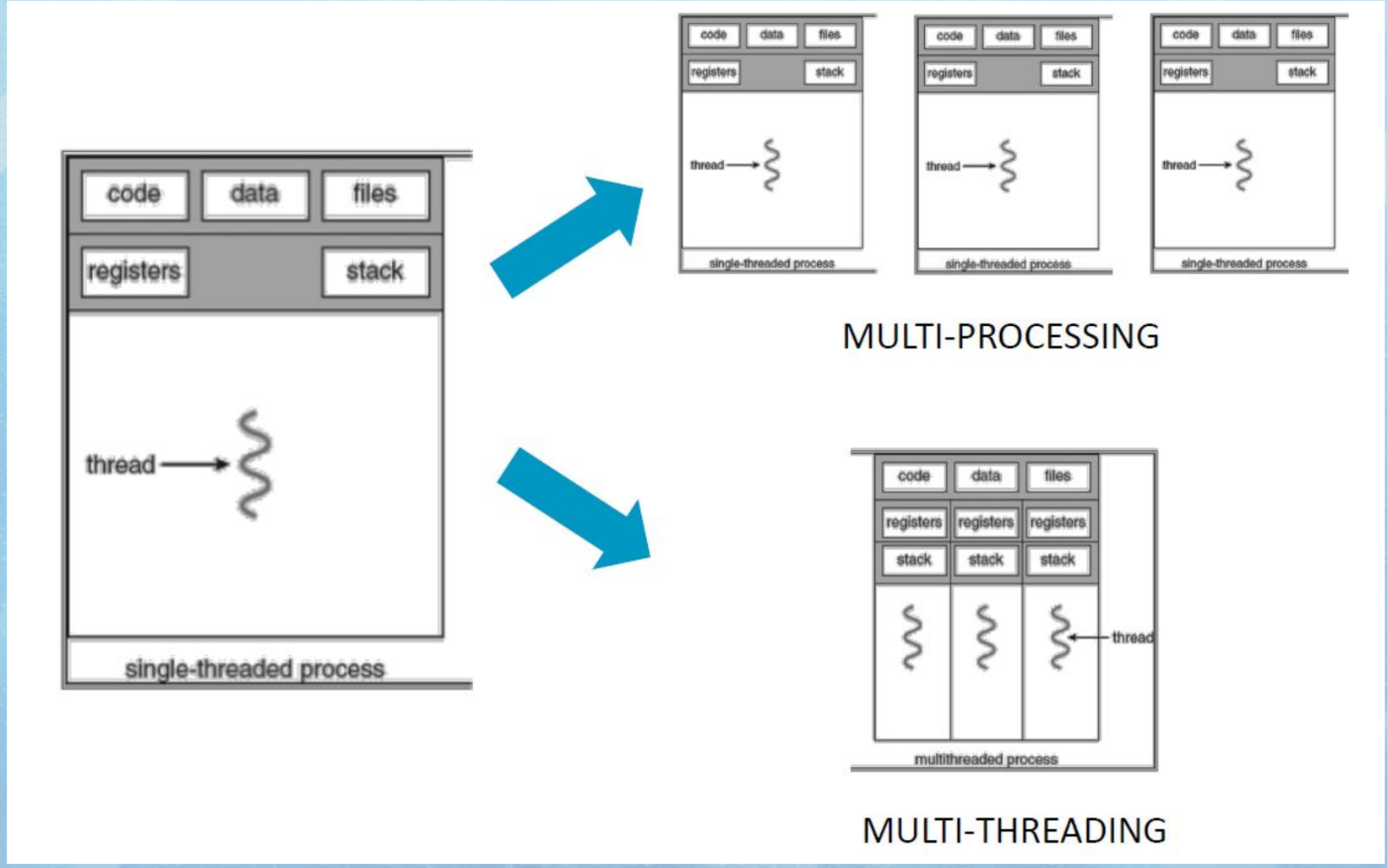
Cpython, GIL and Performance

- Multi-processing Basics - What is a Process?
  - A process is an instance of a computer program that is being executed
  - A process can have 1 or several threads (1 in this hands-on)
  - The kernel of the OS schedule threads to multiple cores



Cpython, GIL and Performance

- Multi-processing Basics - Multi-Processing vs Multi-Threading



Cpython, GIL and Performance

- Multi-processing Basics - Multi-Processing vs Multi-Threading in Python
- The CPython implementation (called CPI, C Python Interpreter) is not thread-safe and only permits a single thread to run at a time
- The standard Python library has two main modules for parallel computing:
  - threading
    - Good for I/O bound tasks
    - Subject to the Global Interpreter Lock (GIL)
  - multiprocessing
    - Circumvents the GIL
    - Useful for parallel computation
  - We will focus on the multiprocessing module

Numpy & Interops

- Why is Numpy not affected by the GIL?

- The GIL only locks Python objects to ensure thread safety. If there is no need to access Python objects, the GIL can be released.
- Numpy uses a C extension implementation that does not require accessing Python objects. Therefore, in the C code implementation, the GIL can be directly released. It is acquired again when there is a need to access Python objects. The C-API provides macros for releasing and acquiring the GIL.

- First things first

- Requirements:
  - CPython 2.7.x, 3.x or PyPy > 5:7, with headers
  - pybind11 package installed (pip install pybind11)
  - Non-ancient compiler (Clang > 3:3, GCC > 4:8, MSVS > 2015)
- Boilerplate (will be omitted in most further examples):

```
#include <pybind11/pybind11.h>

namespace py = pybind11;

PYBIND11_MODULE(example, m) {
    ...
}
```

Numpy & Interops

- Let’s write a module
- Let’s bind a C function that adds two integers:

```cpp
int add(int a, int b) {
    return a + b;
}

PYBIND11_MODULE(myadd, m) {
    m.def("add", &add, "Add two integers.");
}
```

. . . or, C++11 style:

```cpp
PYBIND11_MODULE(myadd, m) {
    m.def("add", [](int a, int b) { return a + b; },
          "Add two integers.");
}
```

Numpy & Interops

- Trying it out

- After the code is compiled, it can be used like a normal Python module:

```
>>> from myadd import add

>>> help(add)
add(arg0: int, arg1: int) -> int

Add two integers.

>>> add(1, 2)
3

>>> add('foo', 'bar')
TypeError: add(): incompatible function arguments.
The following argument types are supported:
    1. (arg0: int, arg1: int) -> int

Invoked with: 'foo', 'bar'
```

Numpy & Interops

- Trying it out

- After the code is compiled, it can be used like a normal Python module:

```
>>> from myadd import add

>>> help(add)
add(arg0: int, arg1: int) -> int

Add two integers.

>>> add(1, 2)
3

>>> add('foo', 'bar')
TypeError: add(): incompatible function arguments.
The following argument types are supported:
    1. (arg0: int, arg1: int) -> int

Invoked with: 'foo', 'bar'
```

Python PYPI

What is PyPI?

PyPI is used to find and install packages for projects. It allows developers to easily manage dependencies and streamline the installation process for their applications.

With PyPI, they can save time and effort by reusing existing code rather than writing everything from scratch.

PyPI is maintained by the Python Software Foundation (PSF) and is free to use for developers and end-users.

It is an essential component of the Python ecosystem and has contributed to the popularity and growth of the language.

In Python, a package is a way to organize related modules, functions, and classes hierarchically.

It provides a structured method to bundle code and can be thought of as a directory containing one or more Python modules.

It helps organize code and makes it easier to manage and distribute.

Packages typically include an init.py file that is executed when the package is imported.

It can define package-level attributes, functions, and classes that can be accessed by other modules in the package.

The init.py file can also include import statements to import modules and functions from other packages.

Python packages can be installed and managed using package managers like pip which can automatically download and install required dependencies.

With packages, developers can encapsulate related functionality into a single unit, making it easier to reuse and maintain code.

For example, the popular NumPy package provides functionality for working with arrays and matrices

while the Pandas package provides tools for data analysis and manipulation.

PyPI: A pillar for Python projects

PyPI has a vast collection of libraries, frameworks, and tools that can be installed and integrated into Python projects.

Here are some of the benefits:

1. Easy package discovery: Provides a user-friendly website that allows developers to search for and discover new packages that they can use in their projects.

2. Package installation: Makes it easy to install packages using the pip command, which is the de facto standard package installer for Python.

This lets developers quickly install and use third-party packages.

3. Package versioning: Maintains a record of all package versions, allowing developers to install and use specific versions of a package.

4. Package dependencies: Allows packages to specify their dependencies on other packages.

This ensures that all required packages are installed when a developer installs a particular package.

5. Community support: Maintained by the Python community. It has a vibrant ecosystem of contributors who help maintain and update packages. This means that developers can rely on the community to provide support and updates for the packages they use.

Using PyPI can save developers time and effort by providing access to pre-built functionality that they can incorporate into projects, leading to faster development time and more robust software applications.

Components of a package

The main components of a Python package are:

1. Package directory: Contains the package's modules, including the special init.py file.

2. init.py: A special Python file that is executed when the package is imported.

It can contain code to initialize the package as well as definitions for variables, functions, and classes that are accessible to other modules within the package.

3. Module: A Python file that contains code that can be executed or imported by other modules.

A package can contain one or more modules.

4. Subpackages: A package can contain one or more sub-packages which are themselves packages that contain their own modules and subpackages.

5. Setup file: Defines metadata about the package, such as its name, version, author, and dependencies.

This file is used by package managers like pip to install the package and its dependencies.

6. README file: Provides information about the package,

Overall, the components of a Python package work together to organize and distribute code in a modular and maintainable way.

Python Internal

Introduction

Python is an interpreted, object-oriented, high-level programming language with dynamic semantics. Python programs are also platform independent.

Once we write a Python program, it can run on any platform without rewriting once again.

Python uses PVM to convert python code to machine understandable code.

Now let us see the internal workings of python, before going in-depth into the technical concepts

Let us try to understand few technical terms to have a better understanding.

Compiler VS Interpreter

An interpreter is a computer program,

which converts each high-level program statement

into the machine code.

This includes source code, pre-compiled code, and scripts.

Both compiler and interpreters do the same job which is

converting higher level programming language to machine

code.

However,

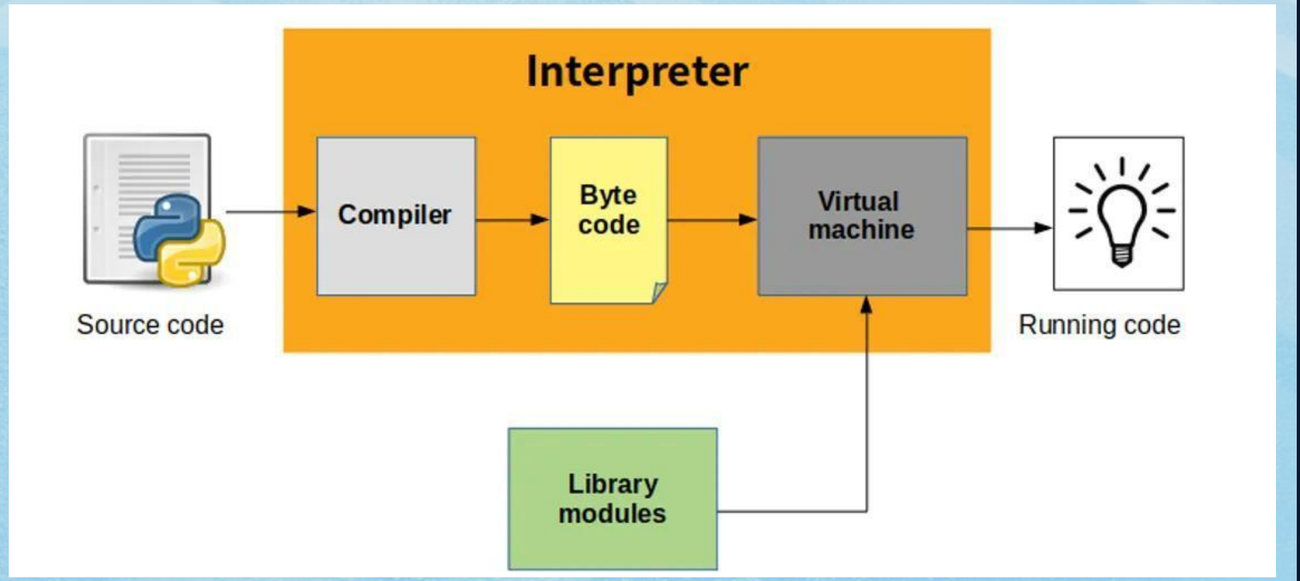
a compiler will convert the code into machine code

before program run.

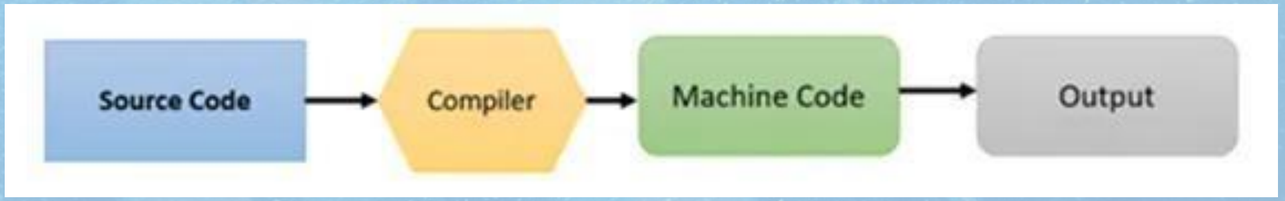
Interpreters convert code into machine code

when the program is run.

Interpreter:



Compiler:



What is PVM?

We know that computers understand only machine code that comprises 1s and 0s.

Since computer understands only machine code, it is imperative that we should convert any program into machine code before it is submitted to the computer for execution.

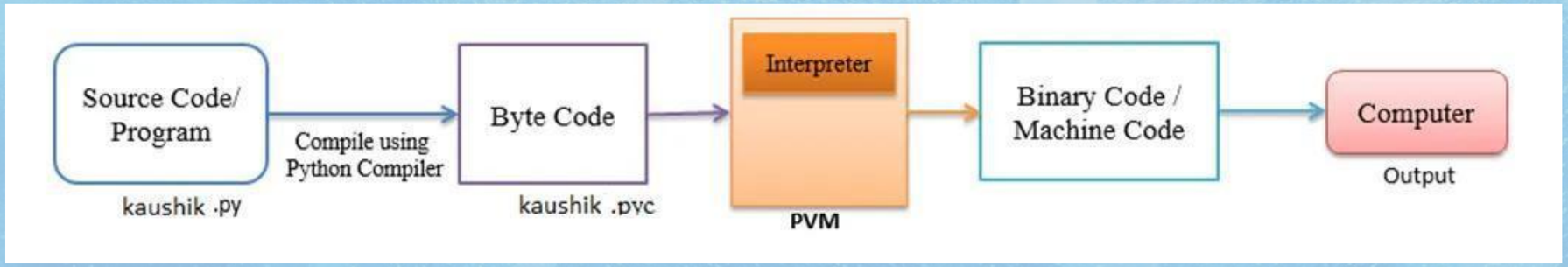
For this purpose, we should take the help of a compiler.

A compiler normally converts the program source code into machine code.

A Python compiler does the same task but in a slightly different manner.

It converts the program source code into another code, called byte code.

Each Python program statement is converted into a group of byte code instructions.
Python Virtual Machine (PVM) takes those byte codes converts those instructions into machine code so that the computer can execute those machine code instructions and display the final output.



To carry out this conversion, PVM is equipped with an interpreter.

The interpreter converts the byte code into machine code and sends that machine code to the computer processor for execution.

Since interpreter is playing the main role, often the Python Virtual Machine is also called an interpreter.

Machine code

Machine code is the low-level binary 1s and 0s that make up the instructions to the processor.

These are processed directly by the CPU and are the final output of a compiler for given CPU and operating system combination.

Machine code for one CPU and OS will not run on different CPU or OS that isn’t compatible.

(i.e. Intel x64 Windows OS machine code will not run on Intel x86 Windows OS).

Byte code

Byte code is a virtualized machine code. Unlike machine code for a real processor,

byte code is often for an idealized or virtual processor that doesn’t actually exist.

Byte code is based on a CPU architecture like a register or stack machine but

often uses general features common to any CPU or instructions and concepts that don’t exist on any CPU.

Python Code:

```
print("My first Blog on Python Internal workings")
a = 10
b = 20
c = a + b
print("c = ",c)
~
```

Byte code:

```
┌──(kaushik㉿kali)-[~]
└─$ python3 -m dis bytecode.py
  1           0 LOAD_NAME                0 (print)
              2 LOAD_CONST               0 ('My first Blog on Python Internal workings')
              4 CALL_FUNCTION            1
              6 POP_TOP

  2           8 LOAD_CONST               1 (10)
             10 STORE_NAME               1 (a)

  3          12 LOAD_CONST               2 (20)
             14 STORE_NAME               2 (b)

  4          16 LOAD_NAME                1 (a)
             18 LOAD_NAME                2 (b)
             20 BINARY_ADD
             22 STORE_NAME               3 (c)

  5          24 LOAD_NAME                0 (print)
             26 LOAD_CONST               3 ('c = ')
             28 LOAD_NAME                3 (c)
             30 CALL_FUNCTION            2
             32 POP_TOP
             34 LOAD_CONST               4 (None)
             36 RETURN_VALUE
┌──(kaushik㉿kali)-[~]
```

After converting high level language to byte code it is sent to python virtual machine to get executed.

Why Interpreted ?

- One popular advantage of interpreted languages is platform independence.
- Python, being an interpreted language, offers this advantage.

-Platform Independence:

- Python bytecode, generated from the source code, can be executed on any platform.
- As long as the bytecode and the Python Virtual Machine (VM) have the same version, the code can run on different platforms.
- This includes platforms like Windows, macOS, Linux, and more.

-Benefits of Platform Independence:

- Simplifies development and deployment across various operating systems.
- Developers can write and test code on one platform and run it on another without major modifications.
- Reduces the need for rewriting or recompiling code for different platforms.

Example:

Developers can write Python code on a Windows machine, generate bytecode, and distribute it to users who run it on macOS or Linux without any compatibility issues.

Memory Management

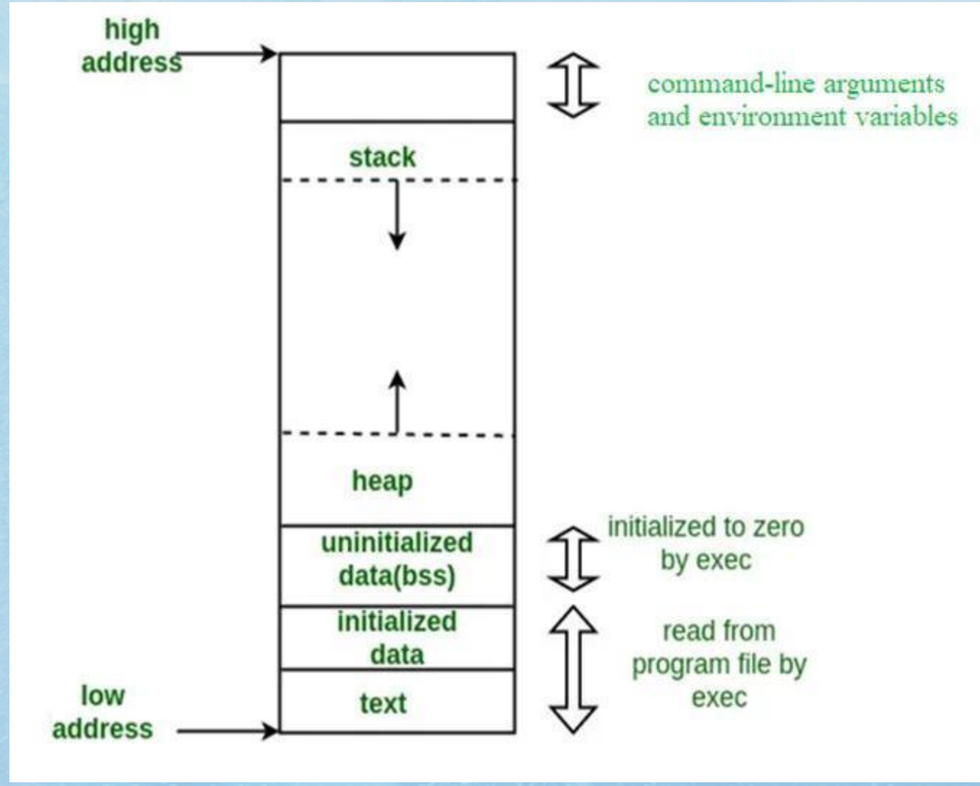
Where is a program stored and executed in computers?

-When you save your program into a file (e.g., 'add.py'), it is automatically stored in the secondary storage (e.g., hard disk).

This storage process is handled by the operating system's kernel, specifically the file management system.

-The Operating System’s kernel (kernel’s File management system) does that.

When we run a program, it is loaded into the main/primary memory of the computer called RAM, the entire program is transferred to the RAM.



Stack

A stack is a special area of computer's memory which stores temporary variables created by a function. In stack, variables are declared, stored and initialized during runtime.

It is a temporary storage memory. When the computing task is complete, the memory of the variable will be automatically erased. The stack section mostly contains methods, local variable, and reference variables.

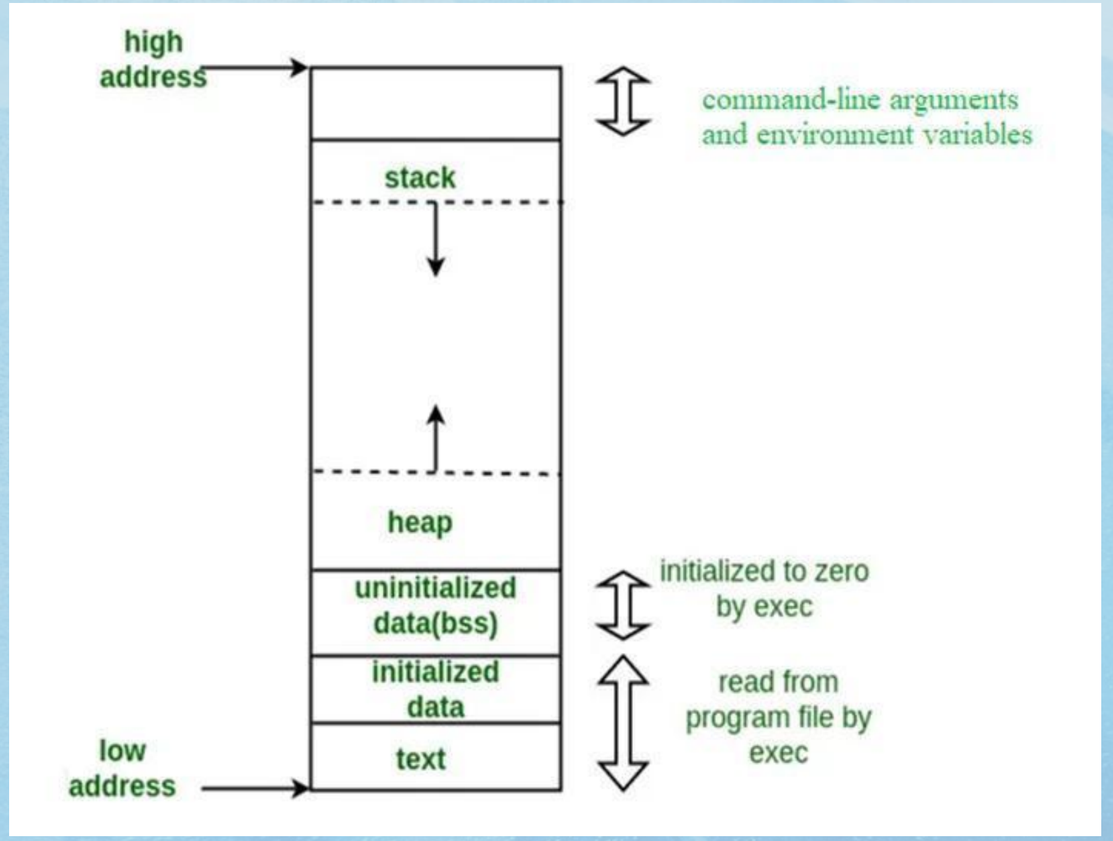
Heap

The heap is a memory used by programming languages to store global variables. By default, all global variable are stored in heap memory space.

It supports Dynamic memory allocation.

The heap is not managed automatically for you

and is not as tightly managed by the CPU.



Is everything in python an object?

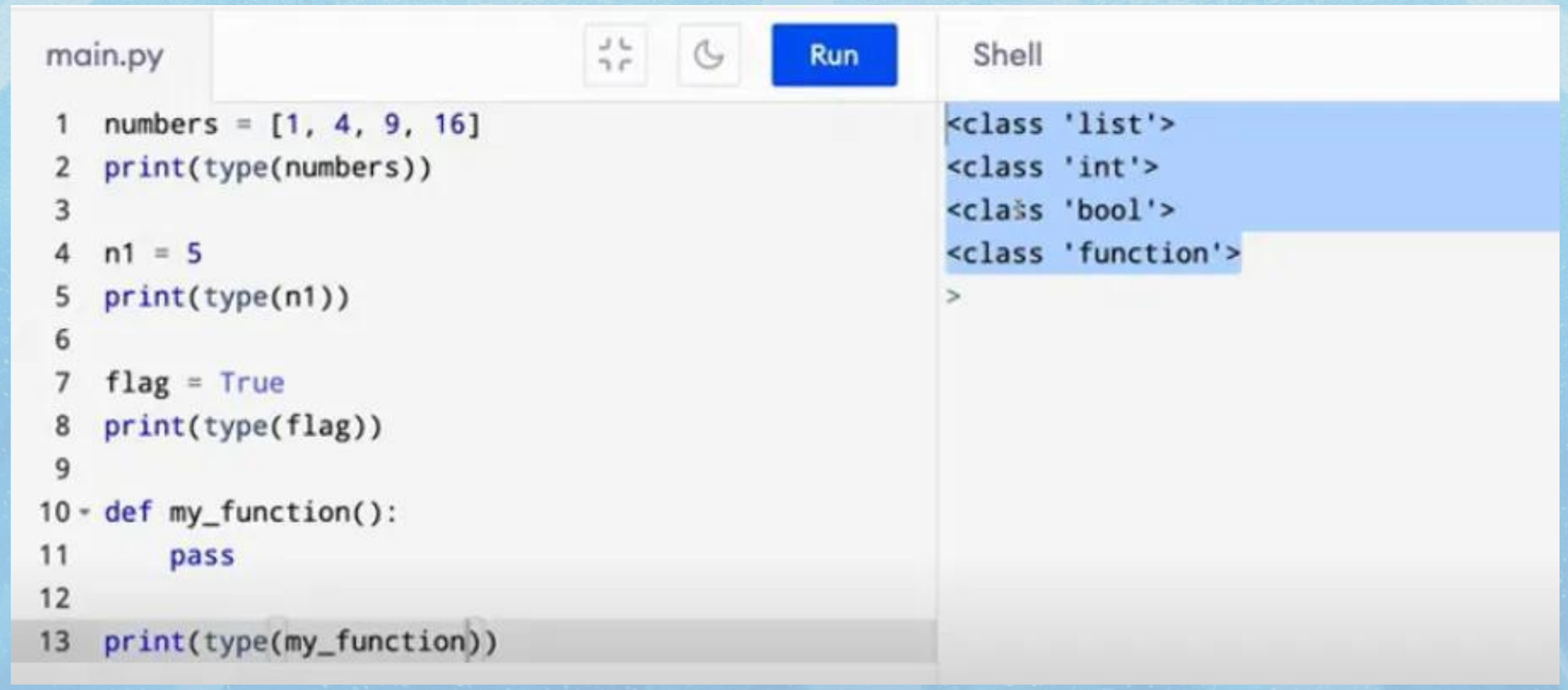
Python is an object-oriented programming language.

Everything is in Python treated as an object, including variable, function, list, tuple, dictionary, set, etc.

Every object belongs to its class.

An object is a real-life entity.

An object is the collection of various data and functions that operate on those data.



Let us see what is reference to have a better understanding of further concepts,

“A link to an object”. A reference is an address that indicates where an object’s variables and methods are stored.

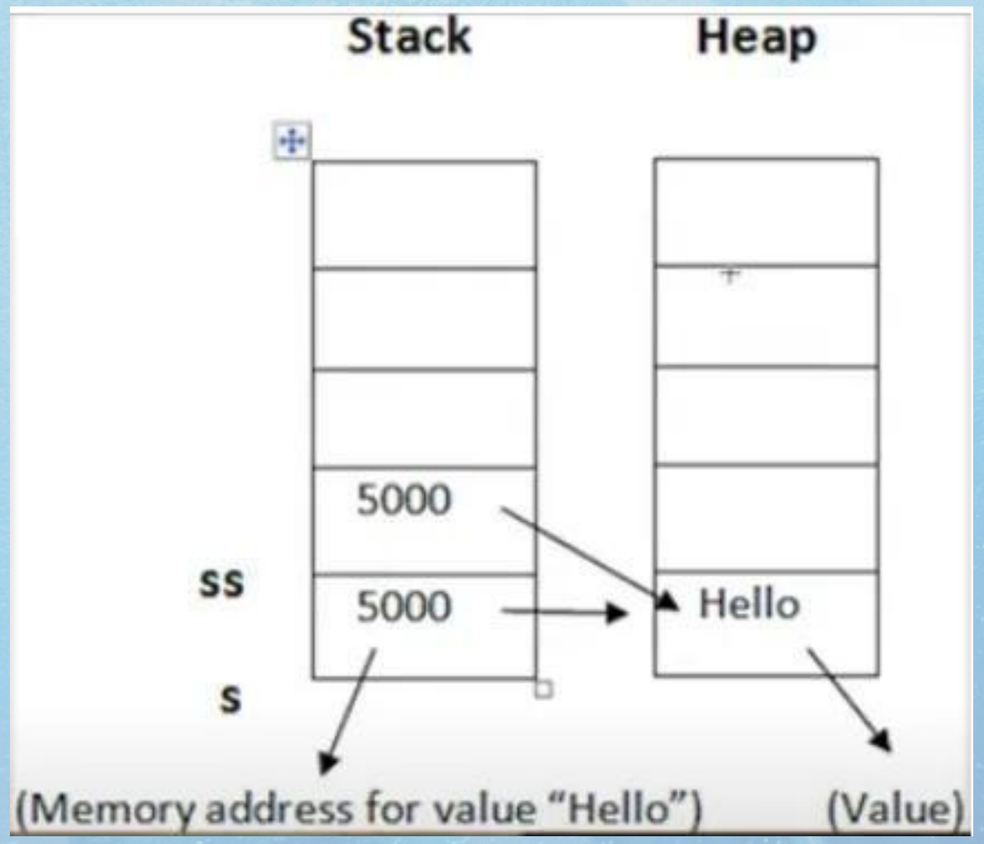
Garbage Collection

Python deletes unwanted objects (built-in types or class instances) automatically to free the memory space. The process by which Python periodically frees and reclaims blocks of memory that no longer are in use is called Garbage Collection.

Code:

```
>>> s = "hello"
>>> ss = "hello"
>>> id(s)
2280292900912
>>> id(ss)
2280292900912
>>>
```

Let us see what happens in the Stack and Heap:



String and References in Memory

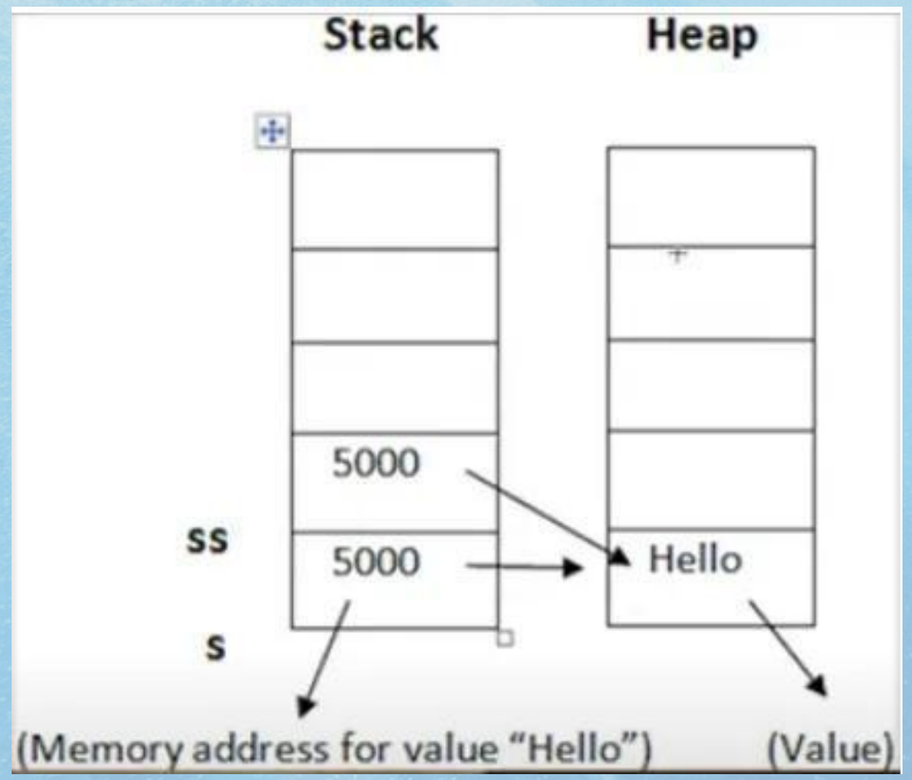
- The string "Hello" is stored in the heap, and a reference is created in the stack to that object.
- The variables "ss" and "s" in the stack have the same memory address, as they both refer to the same object.
- The same concept applies to numbers as well.

-When the value of "s" is changed to "goodbye":

- The value in the heap is not replaced. Instead, a new value is created in the heap, and "s" now refers to the address of the new value.
- The variable "ss" continues to refer to the original value in the heap.

-Reference Count and Garbage Collection:

- Each object in memory has a reference count, which counts the number of variables referring to that object.
- When the reference count becomes less than 1, the object is deleted from memory by the garbage collector.
- The garbage collector deallocates objects in the heap that no longer have references.



For better understanding use memory dump tools to extract and analyse data from RAM.